

Sequenzgebundene REST-Schnittstellen (SC-REST): Ein formales Erweiterungsmuster für die Abbildung von Geschäftsprozessen auf REST-APIs

Stephan Epp

15. Juni 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Problemstellung	3
1.2	Motivation	3
1.3	Beitrag dieser Arbeit	4
2	Grundlagen	4
2.1	REST und Hypermedia	4
2.2	Formale Definitionen	5
3	Das SC-REST-Pattern	6
3.1	Prozessinstanzen und Serverzustand	6
3.2	Zustandslosigkeit und SC-REST	8
3.3	Implementierung der Übergangsprüfung	8
4	Formale Analyse	9
4.1	Korrektheit der Sequenzkontrolle	9
4.2	Laufzeit und Optimalität der Übergangsprüfung	10
4.3	Konformität als Pfad-Subgraph-Problem	11
4.4	Permutierbare optionale Teilschritte	12
4.5	Spekulative Vorbereitung (Prefetching)	14
4.6	Robustheit: Idempotenz und Replay-Sicherheit	16
5	Empirische Veranschaulichung	17
6	Implementierung	18
7	Von SC-REST zu HST: Horizon State Transfer	20
7.1	Motivation: Das Navigationsdefizit von SC-REST	20
7.2	Das HST-Pattern: Formale Definition	21
7.3	Zustandslosigkeit von HST	22
7.4	Größe des Horizont-Subgraphen	23
7.5	Lokale Vollständigkeit und Planungshorizont	24
7.6	Kostenoptimaler Horizont	26

7.7	Versionierung und Cache-Kohärenz via Signaturen	28
7.8	HST als Architekturstil: Generalisierung von REST	29
7.9	Referenzimplementierung: Berechnung des Horizont-Subgraphen	30
8	Zusammenfassung	31
8.1	Anwendungen	32
8.2	Ausblick	32
8.2.1	Formale Spezifikationssprache für Prozessautomaten	32
8.2.2	Verteilte Prozessinstanzen und das Saga-Pattern	33
8.2.3	Nichtdeterministische Automaten und parallele Teilprozesse	33
8.2.4	Sicherheit: Geschützte Zustandstoken	33
8.2.5	Adaptive Vorhersage mittels maschinellem Lernen	34
8.2.6	Verifikation und Model Checking	34
8.2.7	Integration mit GraphQL, gRPC und AsyncAPI	34
8.2.8	Hierarchische Automaten und Subprozesse	34
8.2.9	Standardisierung: Vorschlag für eine OpenAPI-Erweiterung „x-sc-rest“	35
8.2.10	Experimentelle Validierung in Produktionssystemen	35
8.2.11	Energieeffizienz durch reduzierte Fehlversuche	35
8.2.12	Multi-Tenant-Architekturen und verteilte Zustandsspeicher	36
8.2.13	Adaptiver Horizont: Lernen von h^* pro Client und Prozesstyp	36
8.2.14	Informationsfreigabe und Zugriffskontrolle im Horizont-Subgraphen	36
8.2.15	Komposition zu einem globalen Anwendungsgraphen	37
8.2.16	Graphkompression für große Horizonte mittels Signaturen	37
8.2.17	Beziehung zu GraphQL: clientseitig verhandelter Horizont	38
8.2.18	Empirische Validierung des Kostenmodells (α, β, p)	38
8.2.19	Versionsmigration laufender Prozessinstanzen	38
8.3	Implementierungshinweis	39

1 Einführung

REST (*Representational State Transfer*) ist nach Fielding [1] ein Architekturstil, dessen zentrales Prinzip die *Zustandslosigkeit* (*Statelessness*) zwischen aufeinanderfolgenden Anfragen ist: Jede Anfrage muss alle zu ihrer Bearbeitung notwendigen Informationen enthalten, der Server hält keinen Sitzungszustand zwischen Requests vor [2]. Dieses Prinzip ist wesentlich für Skalierbarkeit, Cachebarkeit und die unabhängige Entwicklung von Client und Server.

In der praktischen Anwendung treten jedoch häufig *Geschäftsprozesse* auf, die aus fachlicher Sicht eine *verbindliche Reihenfolge* von API-Aufrufen erfordern: Ein Warenkorb muss erstellt werden, bevor Artikel hinzugefügt werden können; eine Zahlung kann erst nach Abschluss des Checkout-Vorgangs ausgelöst werden; eine Bestellung kann erst bestätigt werden, nachdem die Zahlung autorisiert wurde. Die klassische REST-Architektur trifft hierzu keine normative Aussage: Ein zustandsloser Server kann grundsätzlich nicht unterscheiden, ob eine Anfrage „zur falschen Zeit“ eintrifft, sofern die Anfrage selbst syntaktisch und semantisch valide ist.

1.1 Problemstellung

Gegeben sei ein Geschäftsprozess, der durch eine Menge von API-Operationen $\Sigma = \{\sigma_1, \dots, \sigma_m\}$ (HTTP-Methode und Ressourcenpfad) realisiert wird. Für eine konkrete *Prozessinstanz* (z. B. eine Bestellung, einen Antrag, einen Vertragsabschluss) existiert eine Menge *zulässiger Aufrufsequenzen* $c = (\sigma_{i_1}, \sigma_{i_2}, \dots, \sigma_{i_k}) \in \Sigma^*$.

Ziel: Definiere eine formale, REST-kompatible Erweiterung, mit der ein Server

- jede eingehende Anfrage gegen die für die jeweilige Prozessinstanz aktuell zulässigen Operationen prüft,
- bei Abweichung von der vereinbarten Sequenz eine eindeutige Fehlermeldung liefert, ohne die Zustandslosigkeit auf Protokollebene (jede Anfrage bleibt vollständig selbstbeschreibend) aufzugeben,
- dem Client mittels *Hypermedia as the Engine of Application State* (HATEOAS) jederzeit die Menge der zulässigen Folgeoperationen mitteilt,
- die Kenntnis der wahrscheinlichen Folgeoperation(en) zur *spekulativen Vorbereitung* (Prefetching, Vorberechnung, Caching) nutzt.

1.2 Motivation

Drei Beobachtungen motivieren diese Arbeit:

1. **Führung des Clients.** Ein Client, der eine ungültige Operation versucht, erhält sofort eine maschinenlesbare Fehlermeldung mit der Menge der tatsächlich zulässigen Folgeoperationen. Dies reduziert Implementierungsfehler und Dokumentationsaufwand, da der Prozess *im Protokoll selbst* kodiert ist.
2. **Führung des Servers.** Da der Server zu jedem Zeitpunkt T_{think} des Clients die Menge der *möglichen* Folgezustände $N(s)$ kennt, kann er Ressourcen, Datenbankabfragen oder Berechnungen für diese Zustände bereits während der Bedenkzeit des Clients vorbereiten.

3. **Strukturelle Nähe zum Subgraph-Problem.** Die Prüfung, ob eine beobachtete Aufrufsequenz mit einem vorgegebenen Prozessautomaten konform ist, lässt sich als Spezialfall eines Graph-Enthaltenseins-Problems auffassen. Für Prozessphasen mit *permutierbaren* optionalen Teilschritten erweist sich der in [12] entwickelte Subgraph Algorithmus mit zyklischer Rotation als das strukturell passende und nachweislich optimale Werkzeug.

1.3 Beitrag dieser Arbeit

Diese Arbeit liefert:

- eine formale Definition eines *Prozessautomaten* als Erweiterung des REST-Architekturstils (Abschnitt 2),
- die Definition des *SC-REST-Patterns* (*Sequence-Constrained REST*) mit konkreter HTTP-Semantik, einschließlich eines neuen Problem-Detail-Typs gemäß RFC 7807 [3] (Abschnitt 3),
- den formalen Nachweis von Korrektheit (Soundness/Completeness), Laufzeitkomplexität und Optimalität der Sequenzprüfung (Abschnitt 4),
- eine formale Anbindung permutierbarer optionaler Teilschritte an den Subgraph Algorithmus aus [12], einschließlich eines neuen Zustandsraum-Explosions-Lemmas (Abschnitt 4.4),
- ein formales Latenzmodell für serverseitiges Prefetching mit Beweis des Latenzgewinns (Abschnitt 4.5),
- eine Referenzimplementierung in Python (Abschnitt 6),
- eine Veranschaulichung aller Ergebnisse durch matplotlib-Diagramme,
- einen ausführlichen Ausblick auf Standardisierung, verteilte Prozesse, Verifikation und adaptive Vorhersagemodelle (Abschnitt 8.2).

2 Grundlagen

2.1 REST und Hypermedia

REST-APIs adressieren *Ressourcen* über URIs und manipulieren diese mittels einer einheitlichen Schnittstelle aus HTTP-Methoden (GET, POST, PUT, PATCH, DELETE). Im *Richardson Maturity Model* [5] bildet *HATEOAS* die höchste Stufe: Antworten enthalten Links auf zulässige Folgeoperationen, sodass der Client nicht auf statische, externe Dokumentation angewiesen ist. SC-REST baut unmittelbar auf dieser Stufe auf, erweitert sie jedoch um eine *normative, serverseitig durchgesetzte* Komponente: Die in den Links angegebenen Folgeoperationen sind nicht nur *Vorschläge*, sondern die *einzigen* Operationen, die der Server für die betreffende Prozessinstanz akzeptiert.

2.2 Formale Definitionen

Definition 1 (Operation). Eine *Operation* $\sigma = (M, P)$ besteht aus einer HTTP-Methode $M \in \{\text{GET}, \text{POST}, \text{PUT}, \text{PATCH}, \text{DELETE}\}$ und einem Ressourcenpfad-Muster P . Die Menge aller im Prozess verwendeten Operationen wird mit Σ bezeichnet (das *Operationsalphabet*).

Definition 2 (Prozessautomat). Ein *Prozessautomat* ist ein Tupel

$$P = (S, \Sigma, \delta, s_0, F)$$

mit:

- S : endliche Menge von *Prozesszuständen*,
- Σ : endliches Operationsalphabet,
- $\delta : S \times \Sigma \rightarrow S$: *partielle* (deterministische) Übergangsfunktion,
- $s_0 \in S$: Startzustand,
- $F \subseteq S$: Menge der *Endzustände* (abgeschlossene Prozessinstanzen).

Für $s \in S$ heißt

$$N(s) := \{ \delta(s, \sigma) \mid \sigma \in \Sigma, \delta(s, \sigma) \text{ ist definiert} \}$$

die Menge der *erreichbaren Folgezustände* und

$$\Sigma(s) := \{ \sigma \in \Sigma \mid \delta(s, \sigma) \text{ ist definiert} \}$$

die Menge der in s *zulässigen Operationen*. Die Größe $d(s) := |\Sigma(s)|$ heißt *Verzweigungsgrad* von s .

Definition 3 (Aufrufsequenz und Lauf). Eine *Aufrufsequenz* $c = (c_1, \dots, c_k) \in \Sigma^*$ ist eine endliche Folge von Operationen. Ein *Lauf* von P auf c ist eine Folge von Zuständen $s_0, s_1, \dots, s_k$ mit $s_i = \delta(s_{i-1}, c_i)$ für $i = 1, \dots, k$.

Definition 4 (Konformität). Eine Aufrufsequenz $c = (c_1, \dots, c_k)$ heißt (*partiell*) *konform* bezüglich P , wenn ein Lauf $s_0, \dots, s_k$ gemäß Definition 3 existiert, d. h. wenn $\delta(s_{i-1}, c_i)$ für alle $i = 1, \dots, k$ definiert ist. Die Sequenz heißt *vollständig konform* (oder *abgeschlossen*), wenn zusätzlich $s_k \in F$ gilt. Die Menge aller partiell konformen Sequenzen heißt $\mathcal{L}^*(P)$, die Menge aller vollständig konformen Sequenzen $\mathcal{L}(P) \subseteq \mathcal{L}^*(P)$.

Bemerkung 1. Da δ als *Funktion* (nicht als Relation) definiert ist, ist der Folgezustand $s_i = \delta(s_{i-1}, c_i)$ für jeden Schritt eindeutig bestimmt. Diese Eigenschaft ist notwendig (aber nicht hinreichend), damit ein Server zu jedem Zeitpunkt einen wohldefinierten Prozesszustand pro Prozessinstanz führen kann.

Beispiel 1 (Bestellprozess). Abbildung 1 zeigt einen Prozessautomaten für einen vereinfachten Bestellprozess mit $S = \{s_0, s_1, s_2, s_3, s_4, s_5\}$, $F = \{s_5\}$ und $\Sigma = \{\text{POST /cart}, \text{POST /cart/items}, \text{POST /checkout}, \text{POST /payment}, \text{POST /confirm}\}$. Tabelle 1 listet die Übergangsfunktion δ vollständig auf. Jede Operation, die in einem Zustand s nicht in $\Sigma(s)$ enthalten ist, führt — gemäß der in Abschnitt 3 definierten Semantik — zu einer **Conflict**-Antwort, ohne dass sich der Prozesszustand ändert.

Prozessautomat $P = (S, \Sigma, \delta, s_0, F)$ für einen Bestellprozess

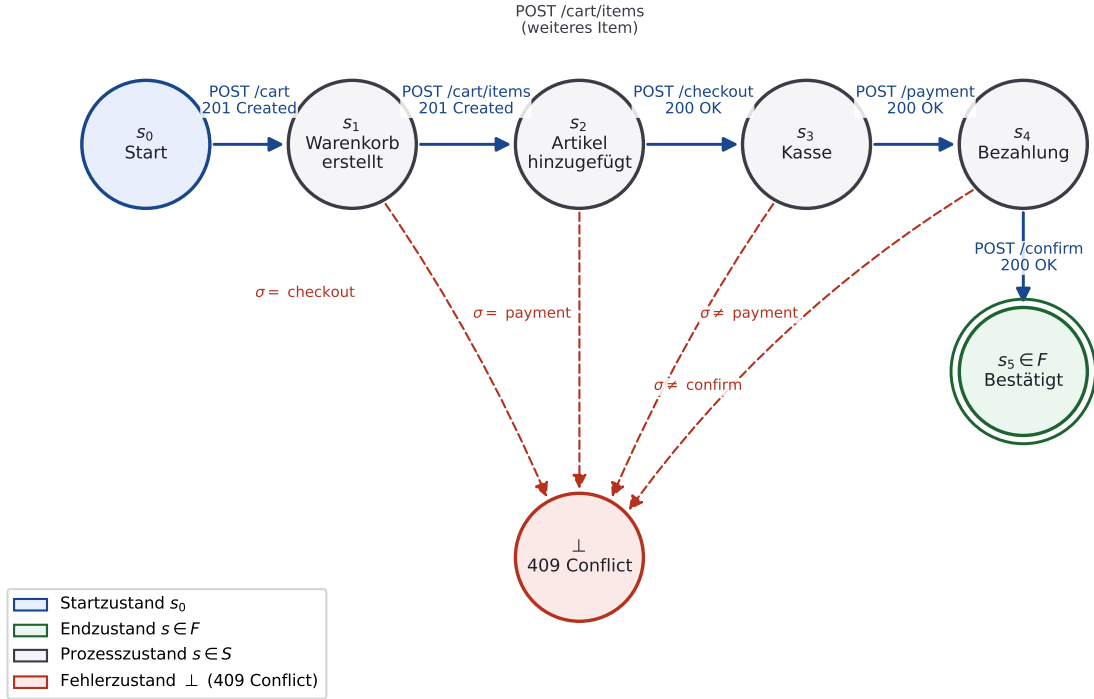


Abbildung 1: Prozessautomat eines Bestellprozesses. Durchgezogene blaue Pfeile bilden den (eindeutigen) Lauf des Hauptpfades; die Selbstschleife an s_2 erlaubt das mehrfache Hinzufügen von Artikeln. Gestrichelte rote Pfeile repräsentieren Operationen, die in dem jeweiligen Zustand *nicht* in $\Sigma(s)$ enthalten sind und daher gemäß Definition 6 mit **409 Conflict** beantwortet werden, ohne den Zustand zu verändern (der Zustand $\perp$ ist daher kein Element von S , sondern eine grafische Sammeldarstellung aller abgewiesenen Übergänge).

Die Sequenz

$$c = (\text{POST /cart}, \text{POST /cart/items}, \text{POST /checkout}, \text{POST /payment}, \text{POST /confirm})$$

ist vollständig konform: Der zugehörige Lauf ist $s_0, s_1, s_2, s_3, s_4, s_5$ mit $s_5 \in F$. Die Sequenz $c' = (\text{POST /cart}, \text{POST /checkout})$ ist *nicht* konform, da $\delta(s_1, \text{POST /checkout})$ undefiniert ist ($\Sigma(s_1) = \{\text{POST /cart/items}\}$).

3 Das SC-REST-Pattern

3.1 Prozessinstanzen und Serverzustand

REST verlangt, dass *jede Anfrage* alle notwendigen Informationen enthält. SC-REST verletzt dieses Prinzip nicht: Der pro Prozessinstanz geführte Zustand $s \in S$ ist kein verstecktes Sitzungsobjekt, sondern wird als *Ressourcenattribut* der Prozessinstanz modelliert und ist über deren URI adressierbar.

Tabelle 1: Übergangsfunktion δ des Bestellprozesses aus Abbildung 1.

Zustand s	Operation σ	Folgezustand $\delta(s, \sigma)$
s_0	POST /cart	s_1
s_1	POST /cart/items	s_2
s_2	POST /cart/items	s_2
s_2	POST /checkout	s_3
s_3	POST /payment	s_4
s_4	POST /confirm	s_5

Definition 5 (Prozessinstanz). Eine *Prozessinstanz* ist ein Tupel (id, s) , wobei id ein eindeutiger Bezeichner (z. B. eine UUID, kodiert in der Ressourcen-URI `/orders/{id}`) und $s \in S$ der aktuelle Zustand der Instanz im Prozessautomaten P ist. Der Server führt eine (partielle) Abbildung $\pi : ID \rightarrow S$ mit $\pi(id) = s$.

Definition 6 (SC-REST-Operationssemantik). Sei $P = (S, \Sigma, \delta, s_0, F)$ ein Prozessautomat und $\pi(id) = s$ der aktuelle Zustand einer Prozessinstanz. Auf eine eingehende Anfrage $\sigma \in \Sigma$ reagiert ein SC-REST-Server wie folgt:

1. **Falls $\delta(s, \sigma)$ definiert ist** (*konformer Übergang*):
 - Führe die durch σ beschriebene Fachlogik aus,
 - setze $\pi(id) := \delta(s, \sigma)$,
 - antworte mit einem 2xx-Statuscode und einer Repräsentation, die gemäß HATEOAS Links auf alle Operationen $\Sigma(\delta(s, \sigma))$ enthält.
2. **Falls $\delta(s, \sigma)$ undefiniert ist** (*Sequenzverletzung*):
 - $\pi(id)$ bleibt unverändert,
 - der Server antwortet mit 409 Conflict und einem Problem-Detail-Objekt gemäß RFC 7807 [3] vom Typ "sequence-violation", das die Menge $\Sigma(s)$ der tatsächlich zulässigen Operationen enthält.

Listing 1: Problem-Detail-Antwort bei Sequenzverletzung (RFC 7807)

```

1 HTTP/1.1 409 Conflict
2 Content-Type: application/problem+json
3
4 {
5   "type": "https://api.example.com/probs/sequence-violation",
6   "title": "Ungueltige Operation fuer aktuellen Prozesszustand",
7   "status": 409,
8   "instance": "/orders/7f3e-...-91ab",
9   "currentState": "s1",
10  "attemptedOperation": "POST /checkout",
11  "allowedOperations": [
12    { "method": "POST", "path": "/orders/7f3e-.../items",
13      "rel": "add-item" }
14  ]
15 }
```

Listing 2: Erfolgsantwort mit HATEOAS-Links auf zulaessige Folgeoperationen

```

1 HTTP/1.1 201 Created
2 Content-Type: application/json
3
4 {
5   "orderId": "7f3e-...-91ab",
6   "state": "s2",
7   "_links": {
8     "add-item": { "method": "POST",
9                   "href": "/orders/7f3e-.../items" },
10    "checkout": { "method": "POST",
11                 "href": "/orders/7f3e-.../checkout" }
12   }
13 }
```

3.2 Zustandslosigkeit und SC-REST

Bemerkung 2. SC-REST hebt die Zustandslosigkeit der einzelnen Anfrage nicht auf. Der Zustand s ist eine *persistente Eigenschaft der adressierten Ressource* (der Prozessinstanz), nicht ein an die Verbindung oder Sitzung gebundenes Artefakt. Jede Anfrage bleibt selbstbeschreibend: Methode, Pfad und Repräsentation sind hinreichend, um die Anfrage zu bearbeiten; lediglich die *Akzeptanz* der Anfrage hängt zusätzlich vom persistenten, ressourcengebundenen Zustand $\pi(id)$ ab — analog zur bekannten Bedingung, dass ein PUT auf eine Ressource mit *If-Match-Header* vom aktuellen ETag der Ressource abhängt. SC-REST verallgemeinert dieses Prinzip von einzelnen Versionskonflikten auf *Sequenzkonflikte*.

3.3 Implementierung der Übergangsprüfung

Algorithmus 3 zeigt die Kernprüfung als Python-Funktion. Die Übergangsfunktion δ wird als verschachteltes dict (Hashtabelle) $\Sigma \times S \rightarrow S$ realisiert, sodass die Prüfung in konstanter erwarteter Zeit erfolgt (siehe Satz 2).

Listing 3: SC-REST Uebergangspruefung als Middleware-Funktion

```

1 def handle_request(process: ProcessAutomaton,
2                   instance_id: str,
3                   operation: tuple) -> Response:
4     """
5     Prueft eine eingehende Operation gegen den aktuellen
6     Zustand einer Prozessinstanz und fuehrt bei Konformitaet
7     den Zustandsuebergang durch.
8
9     delta: Dict[Tuple[State, Operation], State]
10    """
11    current_state = process.pi[instance_id]
12
13    next_state = process.delta.get((current_state, operation))
14
15    if next_state is None:
16        # Sequenzverletzung: Zustand bleibt unveraendert
17        allowed = [op for (s, op) in process.delta
```



```

18         if s == current_state]
19     return problem_detail(
20         status=409,
21         type="sequence-violation",
22         current_state=current_state,
23         attempted=operation,
24         allowed_operations=allowed,
25     )
26
27     # Konformer Uebergang: Fachlogik ausfuehren, Zustand setzen
28     result = process.execute(operation, instance_id)
29     process.pi[instance_id] = next_state
30
31     return success_response(
32         result=result,
33         state=next_state,
34         links=hateoas_links(process, next_state, instance_id),
35     )

```

4 Formale Analyse

Dieser Abschnitt erbringt vier formale Nachweise: (1) Korrektheit der Sequenzkontrolle (Soundness und Completeness), (2) Laufzeit- und Optimalitätsschranken der Übergangsprüfung, (3) die Charakterisierung der Konformitätsprüfung als Pfad-Subgraph-Problem sowie deren Erweiterung auf permutierbare Teilschritte mittels des Subgraph Algorithmus aus [12], und (4) ein formales Latenzmodell für serverseitiges Prefetching.

4.1 Korrektheit der Sequenzkontrolle

Satz 1 (Korrektheit der Sequenzkontrolle). Sei $P = (S, \Sigma, \delta, s_0, F)$ ein Prozessautomat und sei $c = (c_1, \dots, c_k) \in \Sigma^*$ eine Aufrufsequenz, die einem SC-REST-Server gemäß Definition 6 beginnend bei $\pi(id) = s_0$ Operation für Operation präsentiert wird. Dann gilt:

- (a) (*Soundness*) Der Server antwortet auf $c_1, \dots, c_k$ genau dann *niemals* mit **409 Conflict**, wenn $c \in \mathcal{L}^*(P)$ partiell konform ist.
- (b) (*Completeness*) Ist c partiell konform, so stimmt der nach Verarbeitung von c erreichte Serverzustand $\pi(id)$ exakt mit dem eindeutigen Lauf-Endzustand s_k aus Definition 3 überein.
- (c) (*Fehlerlokalisierung*) Ist c *nicht* konform, so existiert ein eindeutiges minimales $i^* \in \{1, \dots, k\}$, sodass $c_1, \dots, c_{i^*-1}$ konform ist, $\delta(s_{i^*-1}, c_{i^*})$ undefiniert ist, und der Server bei genau diesem Schritt – und nur bei diesem – mit **409 Conflict** antwortet, wobei $\pi(id)$ auf s_{i^*-1} verbleibt.

Beweis. Wir führen vollständige Induktion über $i = 0, \dots, k$ und zeigen die Invariante

$$I(i) : \quad \pi(id) \text{ ist nach Verarbeitung von } c_1, \dots, c_i \text{ wohldefiniert und gleich } s_i,$$

solange $c_1, \dots, c_i$ konform ist, andernfalls gleich s_{i^*-1} für das kleinste $i^* \leq i$, für das $\delta(s_{i^*-1}, c_{i^*})$ undefiniert ist.

Induktionsanfang ($i = 0$): $\pi(id) = s_0$ nach Voraussetzung; $I(0)$ gilt trivial, da die leere Sequenz konform ist mit Lauf s_0 .

Induktionsschritt ($i - 1 \rightarrow i$): Nach Induktionsannahme gilt $I(i - 1)$. Zwei Fälle:

- *Fall A*: $c_1, \dots, c_{i-1}$ ist konform und $\pi(id) = s_{i-1}$. Bei Eintreffen von c_i prüft der Server gemäß Definition 6, ob $\delta(s_{i-1}, c_i)$ definiert ist.
 - Ist es definiert, setzt der Server $\pi(id) := \delta(s_{i-1}, c_i) = s_i$ (Definition 3) und antwortet nicht mit 409. Damit ist $c_1, \dots, c_i$ konform und $I(i)$ gilt mit $\pi(id) = s_i$.
 - Ist es undefiniert, antwortet der Server mit 409 Conflict und $\pi(id)$ bleibt bei s_{i-1} . Damit ist $i^* = i$ das gesuchte minimale Indexelement, und $I(i)$ gilt mit $\pi(id) = s_{i^*-1} = s_{i-1}$.
- *Fall B*: $c_1, \dots, c_{i-1}$ ist bereits nicht konform mit Verletzungsindex $i^* \leq i - 1$ und $\pi(id) = s_{i^*-1}$. Da δ in Definition 6 bei einer Sequenzverletzung den Zustand *nicht* verändert, bleibt $\pi(id) = s_{i^*-1}$ auch nach Verarbeitung von c_i – unabhängig davon, ob $\delta(s_{i^*-1}, c_i)$ definiert wäre, denn der Server prüft stets gegen den *aktuellen*, nicht den ursprünglich vorgesehenen Zustand. $I(i)$ gilt also mit demselben i^* .

Aus $I(k)$ folgen unmittelbar (a)–(c): Tritt während der gesamten Verarbeitung von $c_1, \dots, c_k$ kein Fall „undefiniert“ auf, ist c nach Definition 3 konform mit Lauf $s_0, \dots, s_k$ und der Server antwortet nie mit 409 (Soundness, durch Kontraposition von Fall A in beiden Richtungen). Tritt der Fall „undefiniert“ erstmals bei Index i^* auf, so ist dies nach Konstruktion eindeutig und minimal, der Server antwortet exakt einmal – bei i^* – mit 409, und $\pi(id)$ verbleibt für alle $i \geq i^*$ bei s_{i^*-1} (Fehlerlokalisierung, (c)). Im konformen Fall liefert $I(k)$ direkt (b). $\square$

Bemerkung 3. Satz 1(c) ist die formale Grundlage für die in der Einführung geforderte *Führung des Clients*: Der Server lehnt nicht nur ab, sondern tut dies beim *frühestmöglichen* Zeitpunkt einer Sequenzverletzung, und der Zustand der Prozessinstanz bleibt für *beliebig viele* nachfolgende fehlerhafte Anfragen stabil bei s_{i^*-1} – ein Client kann also nach einem 409 beliebig oft erneut die laut `allowedOperations` korrekte Operation versuchen, ohne dass der Prozesszustand durch die fehlgeschlagenen Versuche beeinflusst wird (*Stabilität unter Fehlversuchen*).

4.2 Laufzeit und Optimalität der Übergangsprüfung

Satz 2 (Laufzeit der Übergangsprüfung). Wird δ als Hashtabelle $H : S \times \Sigma \rightarrow S \cup \{\text{undef}\}$ realisiert (Algorithmus 3), so benötigt die Prüfung, ob $\delta(s, \sigma)$ definiert ist, sowie – falls definiert – die Bestimmung von $\delta(s, \sigma)$, im Erwartungswert $O(1)$ Zeit, unabhängig von $|S|$, $|\Sigma|$ und der Länge k der bisherigen Aufrufsequenz.

Beweis. Ein Hashtabellen-Lookup $H[(s, \sigma)]$ erfordert die Berechnung eines Hashwerts über das Paar (s, σ) – beides Werte konstanter Beschreibungsgröße (Zustands-ID bzw. Methode+Pfad-Muster) – sowie den Zugriff auf den entsprechenden Bucket. Beide Operationen sind unter Standardannahmen an die Hashfunktion (gleichverteilte Streuung, Lastfaktor $O(1)$) im Erwartungswert $O(1)$. Die Anzahl $|S|$ der Zustände und $|\Sigma|$ der Operationen beeinflusst lediglich die *Größe* der Hashtabelle, nicht jedoch – bei konstantem Lastfaktor – die erwartete Zugriffszeit. Die Länge k der bisherigen Sequenz geht nicht in die Prüfung ein, da nur der aktuelle Zustand $\pi(id) = s_k$ betrachtet wird (Markov-Eigenschaft des Prozessautomaten, Definition 3). $\square$

Satz 3 (Optimalität der Gesamtprüfung einer Sequenz). Die Prüfung der Konformität einer vollständigen Aufrufsequenz $c = (c_1, \dots, c_k)$ gemäß Definition 4 benötigt $\Theta(k)$ Zeit und ist asymptotisch optimal: Kein korrekter Algorithmus kann diese Prüfung in $o(k)$ Zeit durchführen.

Beweis. Obere Schranke $O(k)$. Nach Satz 2 kostet jeder der k Übergangsschritte $O(1)$, in Summe also $O(k)$.

Untere Schranke $\Omega(k)$. Sei $\mathcal{A}$ ein beliebiger korrekter Algorithmus, der für eine Eingabesequenz $c = (c_1, \dots, c_k)$ entscheidet, ob $c \in \mathcal{L}^*(P)$. Angenommen, $\mathcal{A}$ liest ein Element c_j für ein $j \in \{1, \dots, k\}$ nicht. Wir konstruieren zwei Eingaben c und c' , die sich nur in c_j unterscheiden:

$$c = (c_1, \dots, c_{j-1}, \sigma, c_{j+1}, \dots, c_k), \quad c' = (c_1, \dots, c_{j-1}, \sigma', c_{j+1}, \dots, c_k),$$

wobei $\sigma, \sigma' \in \Sigma(s_{j-1})$ mit $\sigma \neq \sigma'$ und $\delta(s_{j-1}, \sigma) \neq \delta(s_{j-1}, \sigma')$ – eine solche Wahl existiert, sobald $d(s_{j-1}) \geq 2$, was o. B. d. A. für mindestens ein j gilt (sonst ist P trivial, da jeder Zustand höchstens einen Folgezustand besitzt, und die Aussage wird leer). Für geeignete Fortsetzungen $c_{j+1}, \dots, c_k$ – etwa eine konforme Fortsetzung ab $\delta(s_{j-1}, \sigma)$, die ab $\delta(s_{j-1}, \sigma')$ *nicht* konform ist – unterscheiden sich die korrekten Antworten für c und c' . Da $\mathcal{A}$ den Eintrag c_j nicht liest, verhält sich $\mathcal{A}$ auf c und c' identisch und liefert für mindestens eine der beiden Eingaben ein falsches Ergebnis – Widerspruch zur Korrektheit. Folglich muss $\mathcal{A}$ jedes Element $c_1, \dots, c_k$ lesen, also $T_{\mathcal{A}}(k) \geq \Omega(k)$. $\square$

Bemerkung 4. Satz 3 zeigt, dass die SC-REST-Konformitätsprüfung – anders als das allgemeine Subgraph-Problem aus [12] mit seiner unteren Schranke $\Omega(n^{3-\epsilon})$ – *linear* und damit vergleichsweise „billig“ ist. Der Grund liegt in der *linearen Struktur* einer Aufrufsequenz: Sie ist ein einfacher Pfad, kein beliebiger Graph. Abschnitt 4.4 zeigt, dass diese Eigenschaft verloren geht, sobald *permutierbare* Teilschritte zugelassen werden – und genau dann wird der Subgraph Algorithmus mit zyklischer Rotation wieder relevant.

4.3 Konformität als Pfad-Subgraph-Problem

Definition 7 (Prozessgraph). Der *Prozessgraph* $G_P = (S, E)$ zu einem Prozessautomaten $P = (S, \Sigma, \delta, s_0, F)$ ist der gerichtete, kantenbeschriftete Graph mit

$$E = \{ (s, s', \sigma) \mid s, s' \in S, \sigma \in \Sigma, \delta(s, \sigma) = s' \}.$$

Eine Aufrufsequenz $c = (c_1, \dots, c_k)$ induziert den *Sequenzgraphen* G_c : einen einfachen, gerichteten Pfad mit $k + 1$ Knoten $u_0, \dots, u_k$ und Kanten (u_{i-1}, u_i, c_i) für $i = 1, \dots, k$.

Satz 4 (Pfad-Charakterisierung der Konformität). Eine Aufrufsequenz c ist genau dann partiell konform bezüglich P (Definition 4), wenn der Sequenzgraph G_c unter der Abbildung $u_0 \mapsto s_0$ als beschrifteter Teilpfad in G_P einbettbar ist, d. h. wenn eine Folge von Knoten $s_0 = v_0, v_1, \dots, v_k \in S$ existiert mit $(v_{i-1}, v_i, c_i) \in E$ für alle $i = 1, \dots, k$.

Beweis. „ $\Rightarrow$ “: Ist c konform, existiert nach Definition 3 ein Lauf $s_0, \dots, s_k$ mit $\delta(s_{i-1}, c_i) = s_i$. Nach Definition 7 ist dann $(s_{i-1}, s_i, c_i) \in E$ für alle i , also bildet $v_i := s_i$ die geforderte Pfadeinbettung.

„ $\Leftarrow$ “: Existiert eine solche Knotenfolge $v_0 = s_0, \dots, v_k$ mit $(v_{i-1}, v_i, c_i) \in E$, so gilt nach Definition 7 $\delta(v_{i-1}, c_i) = v_i$ für alle i . Da δ eine Funktion ist (Definition 2), ist diese Folge der *eindeutige* Lauf von P auf c mit $s_i = v_i$, also ist c konform. $\square$

Bemerkung 5. Satz 4 reduziert die Konformitätsprüfung auf die Einbettbarkeit eines *einfachen, beschrifteten Pfades* G_c in G_P – ein Spezialfall des allgemeinen Subgraph-Enthaltenseins-Problems. Während das allgemeine Problem (zwei beliebige Graphen, Einbettung unter Knotenpermutation bzw. zyklischer Rotation) nach [12] eine scharfe untere Schranke $\Omega(n^{3-\varepsilon})$ unter SETH besitzt, ist die Pfad-Einbettung *ab dem festen Startknoten* s_0 wegen der Determiniertheit von δ trivial in $\Theta(k)$ entscheidbar (Satz 3): An jedem Knoten v_i existiert höchstens *eine* mit c_{i+1} beschriftete ausgehende Kante, sodass keine Suche über mehrere Kandidaten und keine Rotation erforderlich ist. Der allgemeine, schwere Fall des Subgraph-Problems tritt erst dann wieder auf, wenn der Startknoten *nicht fest* ist oder wenn – wie in Abschnitt 4.4 – mehrere Kanten mit unterschiedlichen Beschriftungen in beliebiger Reihenfolge durchlaufen werden dürfen.

4.4 Permutierbare optionale Teilschritte

Viele Geschäftsprozesse enthalten Phasen, in denen mehrere optionale Teilschritte *in beliebiger Reihenfolge*, jedoch jeweils höchstens einmal, ausgeführt werden dürfen, bevor der Prozess fortgesetzt wird – etwa das Hinzufügen mehrerer unabhängiger Zusatzoptionen zu einer Bestellung (Geschenkverpackung, Expresslieferung, Gutscheincode, Versicherung), deren Reihenfolge fachlich irrelevant ist.

Definition 8 (Sammelzustand mit permutierbaren Teilschritten). Ein Zustand $s_c \in S$ heißt *Sammelzustand mit r permutierbaren optionalen Teilschritten*, wenn ein Operationssatz $\Sigma_{\text{opt}} = \{\sigma_1, \dots, \sigma_r\} \subseteq \Sigma$ sowie eine Abschlussoperation $\sigma_{\text{done}} \in \Sigma$ existieren, sodass für jede Teilmenge $T \subseteq \{1, \dots, r\}$ der bereits ausgeführten Operationen und jedes $j \notin T$ gilt: Aus dem zu T korrespondierenden (logischen) Zwischenzustand ist σ_j zulässig und führt zum Zwischenzustand $T \cup \{j\}$; aus dem Zwischenzustand zu $T = \{1, \dots, r\}$ (alle Teilschritte ausgeführt, oder ein beliebiges zulässiges T , falls σ_{done} auch früher erlaubt ist) führt σ_{done} in den Folgezustand des Prozesses.

Lemma 1 (Zustandsraum-Explosion bei naiver Automatenkodierung). Sei s_c ein Sammelzustand mit r permutierbaren optionalen Teilschritten gemäß Definition 8, und seien für je zwei verschiedene Teilmengen $T \neq T' \subseteq \{1, \dots, r\}$ die jeweils zulässigen Restoperationen $\Sigma_{\text{opt}} \setminus T$ und $\Sigma_{\text{opt}} \setminus T'$ verschieden (d. h. $\Sigma(s_T) \neq \Sigma(s_{T'})$ für die zu T bzw. T' gehörigen Zwischenzustände $s_T, s_{T'}$). Dann benötigt *jeder* deterministische Prozessautomat, der diesen Sammelzustand abbildet, mindestens 2^r Zustände, also $|S| \geq 2^r$.

Beweis. Wir zeigen, dass die 2^r Zwischenzustände s_T , $T \subseteq \{1, \dots, r\}$, paarweise *verschieden* sein müssen – ein Argument in der Tradition des Myhill-Nerode-Theorems für reguläre Sprachen, angewandt auf Erreichbarkeits-Äquivalenzklassen von Zuständen.

Angenommen, $s_T = s_{T'}$ für zwei verschiedene Teilmengen $T \neq T'$. Da δ als Funktion eines einzelnen Zustands definiert ist (Definition 2), folgt $\Sigma(s_T) = \Sigma(s_{T'})$ – die Menge der von s_T aus zulässigen Operationen hängt nur vom Zustand selbst ab, nicht von der „Historie“, über die er erreicht wurde. Dies widerspricht jedoch der Voraussetzung $\Sigma(s_T) \neq \Sigma(s_{T'})$ für $T \neq T'$.

Also müssen alle 2^r Teilmengen $T \subseteq \{1, \dots, r\}$ auf paarweise verschiedene Zustände $s_T \in S$ abgebildet werden, woraus $|S| \geq 2^r$ folgt. $\square$

Bemerkung 6. Lemma 1 ist die formale Begründung dafür, dass ein SC-REST-Server für Sammelzustände *nicht* naiv δ über den vollständigen Potenzmengen-Zustandsraum $2^{\{1, \dots, r\}}$ kodieren sollte: Bereits für $r = 20$ permutierbare Teilschritte ergäbe dies über

10^6 Zustände allein für diese eine Prozessphase. Stattdessen genügt es, gemäß Satz 5 eine kompakte Repräsentation der Größe $O(r)$ zu führen und die Konformität einer beobachteten Teilreihenfolge mittels des Subgraph Algorithmus zu prüfen.

Definition 9 (Signaturfolgen für Sammelzustände). Für einen Sammelzustand s_c mit Operationssatz $\Sigma_{\text{opt}} = \{\sigma_1, \dots, \sigma_r\}$ sei $\sigma^P = [\sigma_1, \dots, \sigma_r]$ die *kanonische Reihenfolge* (z. B. die im Prozessmodell dokumentierte Referenzreihenfolge). Eine vom Client beobachtete Ausführungsreihenfolge $c' = (c'_1, \dots, c'_r)$ ist eine Permutation von Σ_{opt} . Wir bezeichnen mit $\sigma^C = [c'_1, \dots, c'_r]$ die zugehörige *Beobachtungsfolge*.

Satz 5 (Verifikation permutierbarer Teilschritte via Subgraph Algorithmus). Sei s_c ein Sammelzustand mit r permutierbaren optionalen Teilschritten und sei $c' = (c'_1, \dots, c'_r)$ eine vom Client vollständig durchlaufene Beobachtungsfolge (jede Operation aus Σ_{opt} genau einmal, in beliebiger Reihenfolge). Dann gilt:

- (a) Die Aussage „ c' ist eine zulässige Reihenfolge (Permutation) der Operationen aus Σ_{opt} “ ist äquivalent zur Aussage „die Signaturfolge σ^C ist – als Multimenge – identisch zu σ^P , d. h. σ^C entsteht aus σ^P durch Anwendung einer Permutation auf die Indexmenge $\{1, \dots, r\}$ “.
- (b) Beschränkt man die zulässigen Umordnungen auf *zyklische Rotationen* von σ^P (z. B. weil eine Teilmenge der r Teilschritte eine vom Prozessdesign vorgegebene relative Ordnung beibehalten muss, die nur als Ganzes zyklisch verschoben werden darf), so lässt sich die Prüfung von (a) mit dem Subgraph Algorithmus aus [12] in $\Theta(r^3)$ Zeit durchführen, und diese Laufzeit ist nach dem Optimalitätssatz von [12] asymptotisch optimal unter SETH: Keine Prüfung ist in $O(r^{3-\varepsilon})$ Zeit für $\varepsilon > 0$ möglich.
- (c) Im allgemeinen Fall (beliebige Permutationen statt zyklischer Rotationen) genügt bereits eine Sortierung der Signaturfolgen in $O(r \log r)$ Zeit, um (a) zu entscheiden.

Beweis. (a) Da jede Operation aus Σ_{opt} in c' nach Voraussetzung genau einmal vorkommt, ist c' per Definition eine Permutation von $(\sigma_1, \dots, \sigma_r)$. Die zugehörige Signaturfolge σ^C entsteht somit aus $\sigma^P = [\sigma_1, \dots, \sigma_r]$ durch Umordnung gemäß genau dieser Permutation. Umgekehrt definiert jede Umordnung von σ^P , die als Multimenge mit σ^P übereinstimmt, eindeutig eine Permutation von $\{1, \dots, r\}$ und damit eine zulässige Ausführungsreihenfolge im Sinne von Definition 8 (jede Operation wird, unabhängig von der Reihenfolge, genau einmal ausgeführt und überführt den Sammelzustand in den Folgezustand des Prozesses).

(b) Wende den Subgraph Algorithmus aus [12] auf die beiden Sequenzen σ^P und σ^C an: Beide haben Länge r , die Spalten-Signaturen $\sigma_j = \sum_i A_{ij} 2^i + j \cdot 2^r$ werden für beide Folgen gebildet (hier interpretiert als $1 \times r$ -, „Adjazenzmatrizen“ über dem Operationsalphabet, wobei $A_{ij} = 1$ gdw. die i -te Operation an Position j der jeweiligen Folge steht). Für jede der r zyklischen Rotationen von σ^C wird via LCS geprüft, ob $\sigma^P \subseteq \text{rot}_k(\sigma^C)$ gilt (Satz 2 in [12]). Nach Satz „Laufzeit“ und Satz „Optimalität des Subgraph Algorithmus“ aus [12] ist diese Prüfung in $\Theta(r^3)$ durchführbar und unter SETH nicht in $O(r^{3-\varepsilon})$ für $\varepsilon > 0$ verbesserbar – die untere Schranke aus [12] (kombiniert aus Eingabe-Leseschranke, Notwendigkeit aller Rotationen und der SETH-basierten LCS-Schranke) übertragen sich unverändert, da die Signaturkonstruktion injektiv ist (Lemma „Eindeutigkeit der Signaturen“, [12]) und somit keine Information verloren geht.

(c) Da Signaturen nach [12] (Lemma „Eindeutigkeit der Signaturen“) injektiv und damit als Schlüssel für eine Ordnungsrelation geeignet sind, kann man σ^C und σ^P jeweils

in $O(r \log r)$ sortieren und elementweise vergleichen: Gleichheit der sortierten Folgen ist äquivalent zur Multimengen-Gleichheit aus (a). Dies ist günstiger als $\Theta(r^3)$, jedoch nur anwendbar, wenn *beliebige* Permutationen zulässig sind und keine relative Ordnung zwischen Teilgruppen der r Operationen erhalten bleiben muss. $\square$

Bemerkung 7. Satz 5 zeigt eine doppelte Erweiterung gegenüber [12]: Erstens wird der Subgraph Algorithmus von seinem ursprünglichen Anwendungsfeld (AST-Vergleich, Graph-Deduplizierung) auf die Konformitätsprüfung permutierbarer API-Teilschritte *übertragen*. Zweitens zeigt Lemma 1 zusammen mit Satz 5, dass die signaturbasierte Prüfung eine *exponentielle* Raumersparnis erzielt: Statt $\Omega(2^r)$ Zuständen im naiv kodierten Automaten genügt eine Signaturfolge der Größe $O(r)$ zuzüglich einer Prüfung in $\Theta(r^3)$ – ein klassischer Zeit-Raum-Trade-off, der für praktisch relevante r (typischerweise $r \leq 20$, siehe Abbildung 2) deutlich zugunsten der signaturbasierten Methode ausfällt, da $2^{20} \approx 10^6 \gg 20^3 = 8000$.

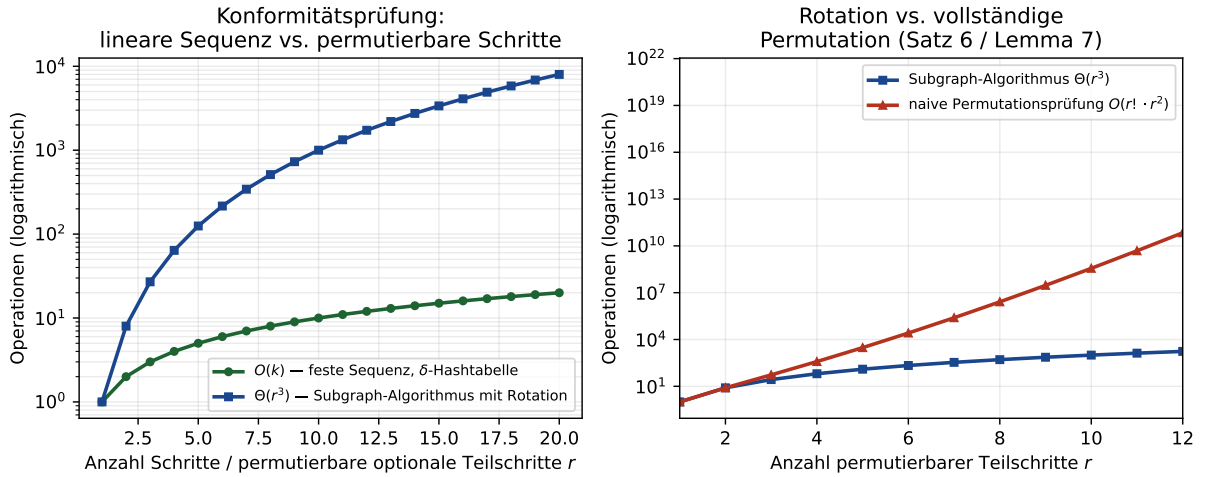


Abbildung 2: Links: Vergleich der Konformitätsprüfung einer festen, linearen Aufrufsequenz ($O(k)$, Satz 3) mit der signaturbasierten Prüfung permutierbarer Teilschritte mittels Subgraph Algorithmus und zyklischer Rotation ($\Theta(r^3)$, Satz 5). Beide Kurven wachsen polynomial, jedoch mit unterschiedlichem Grad – die lineare Sequenzprüfung bleibt für alle betrachteten Größen um Größenordnungen günstiger, was die in Bemerkung 5 beschriebene strukturelle Vereinfachung widerspiegelt. Rechts: Vergleich des Subgraph-Algorithmus ($\Theta(r^3)$) mit der naiven vollständigen Permutationsprüfung ($O(r! \cdot r^2)$, vgl. [12]) auf logarithmischer Skala für $r = 1, \dots, 12$. Bereits ab $r \approx 6$ übersteigt die naive Methode die rotationsbasierte Methode um mehrere Größenordnungen, was Lemma 1 und Bemerkung 7 empirisch illustriert.

4.5 Spekulative Vorbereitung (Prefetching)

Da der Server gemäß Definition 2 für jeden Zustand s die Menge $N(s)$ der erreichbaren Folgezustände *vollständig kennt*, kann er bereits während der Bedenkzeit des Clients Vorbereitungsarbeiten für alle $s' \in N(s)$ anstoßen.

Definition 10 (Latenzmodell). Für eine Operation σ mit $\delta(s, \sigma) = s'$ seien:

- $T_{\text{exec}}(\sigma)$: die Zeit zur Ausführung der eigentlichen Fachlogik von σ (z. B. Schreiboperation, Validierung),
- $T_{\text{prep}}(s')$: die Zeit, um Ressourcen für den Zustand s' *vorzubereiten* (z. B. Vorab-Laden von Referenzdaten, Reservierung von Inventar, Aufwärmen von Caches),
- T_{think} : die Zeit zwischen dem Eintreffen der Antwort auf die vorherige Operation (die in Zustand s überführt hat) und dem Eintreffen der nächsten Anfrage σ des Clients (*Bedenkzeit*).

Ohne Prefetching beginnt die Vorbereitung von s' erst, nachdem σ eingetroffen ist:

$$E[L_{\text{ohne}}] = T_{\text{prep}}(s') + T_{\text{exec}}(\sigma).$$

Mit Prefetching startet der Server, sobald $\pi(id) = s$ erreicht ist, für *jedes* $s'' \in N(s)$ parallel eine Vorbereitung. Trifft σ mit $\delta(s, \sigma) = s'$ ein, ist die Vorbereitung für s' bereits $\min(T_{\text{think}}, T_{\text{prep}}(s'))$ weit fortgeschritten:

$$E[L_{\text{pf}}] = \max(T_{\text{prep}}(s') - T_{\text{think}}, 0) + T_{\text{exec}}(\sigma).$$

Satz 6 (Latenzgewinn durch Prefetching). Unter dem Latenzmodell aus Definition 10 gilt für den Latenzgewinn $\Delta := E[L_{\text{ohne}}] - E[L_{\text{pf}}]$:

$$\Delta = \min(T_{\text{think}}, T_{\text{prep}}(s')) \geq 0.$$

Insbesondere wird die Vorbereitungszeit $T_{\text{prep}}(s')$ *vollständig* durch Prefetching verborgen, sobald $T_{\text{think}} \geq T_{\text{prep}}(s')$.

Beweis. Wir unterscheiden zwei Fälle.

Fall 1: $T_{\text{think}} \geq T_{\text{prep}}(s')$. Dann ist $T_{\text{prep}}(s') - T_{\text{think}} \leq 0$, also $\max(T_{\text{prep}}(s') - T_{\text{think}}, 0) = 0$ und somit $E[L_{\text{pf}}] = T_{\text{exec}}(\sigma)$. Es folgt

$$\Delta = (T_{\text{prep}}(s') + T_{\text{exec}}(\sigma)) - T_{\text{exec}}(\sigma) = T_{\text{prep}}(s') = \min(T_{\text{think}}, T_{\text{prep}}(s')).$$

Fall 2: $T_{\text{think}} < T_{\text{prep}}(s')$. Dann ist $T_{\text{prep}}(s') - T_{\text{think}} > 0$, also $\max(T_{\text{prep}}(s') - T_{\text{think}}, 0) = T_{\text{prep}}(s') - T_{\text{think}}$ und somit

$$E[L_{\text{pf}}] = (T_{\text{prep}}(s') - T_{\text{think}}) + T_{\text{exec}}(\sigma).$$

Es folgt

$$\Delta = (T_{\text{prep}}(s') + T_{\text{exec}}(\sigma)) - ((T_{\text{prep}}(s') - T_{\text{think}}) + T_{\text{exec}}(\sigma)) = T_{\text{think}} = \min(T_{\text{think}}, T_{\text{prep}}(s')).$$

In beiden Fällen gilt $\Delta = \min(T_{\text{think}}, T_{\text{prep}}(s'))$, und da $T_{\text{think}}, T_{\text{prep}}(s') \geq 0$ ist auch $\Delta \geq 0$. $\square$

Bemerkung 8 (Kosten des Prefetchings). Satz 6 betrachtet nur die *Latenz aus Sicht des Clients*. Die *Kosten* des Prefetchings für den Server skalieren mit dem Verzweigungsgrad $d(s) = |N(s)|$ (Definition 2): Für jeden der $d(s)$ möglichen Folgezustände muss eine Vorbereitung angestoßen werden, von denen höchstens eine tatsächlich genutzt wird (*spekulative Verschwendung*). Ein SC-REST-Server sollte daher

- den Verzweigungsgrad $d(s)$ pro Zustand durch Prozessdesign begrenzen (z. B. $d(s) \leq D_{\text{max}}$ für ein konfigurierbares D_{max}),

- Vorbereitungsoperationen mit geringen Kosten $T_{\text{prep}}(s')$ priorisieren, da nach Satz 6 der Gewinn durch $\min(T_{\text{think}}, T_{\text{prep}}(s'))$ für kleine $T_{\text{prep}}(s')$ ohnehin gering ist, während die *verschwendeten* Ressourcen proportional zu $T_{\text{prep}}(s')$ anfallen,
- idempotente bzw. abbrechbare Vorbereitungsoperationen verwenden, sodass im Fall einer nicht genutzten Vorbereitung kein inkonsistenter Zustand entsteht.

Die empirische Auswertung in Abschnitt 5 (Abbildung 4, rechts) zeigt den kumulierten Effekt des Latenzgewinns über mehrere Prozessinstanzen unter realistischer Trefferquote.

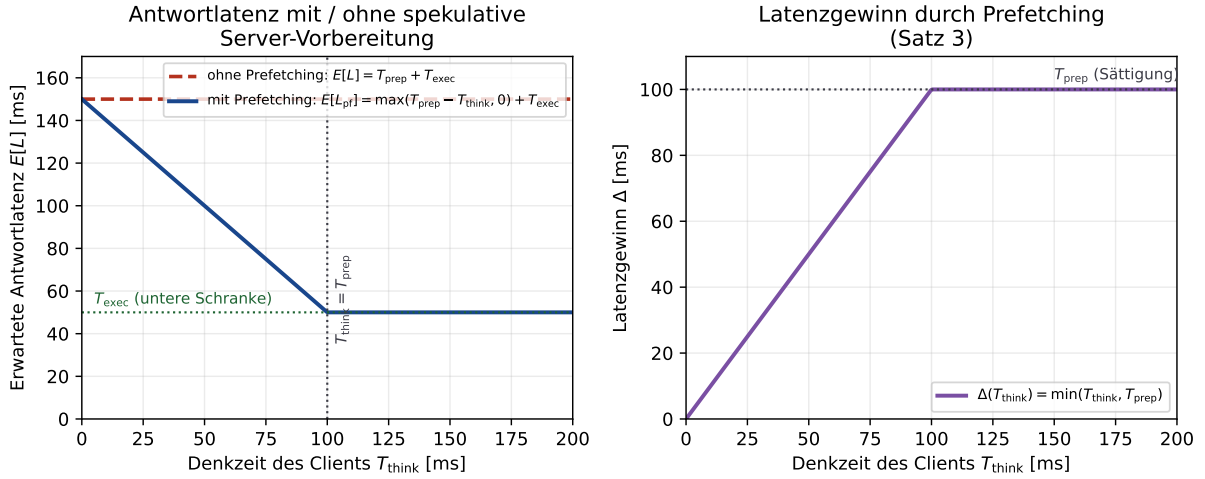


Abbildung 3: Links: Erwartete Antwortlatenz $E[L]$ in Abhängigkeit von der Bedenkzeit T_{think} des Clients, für $T_{\text{prep}} = 100$ ms und $T_{\text{exec}} = 50$ ms. Ohne Prefetching (rot, gestrichelt) ist die Latenz konstant $T_{\text{prep}} + T_{\text{exec}} = 150$ ms, unabhängig von T_{think} . Mit Prefetching (blau) sinkt die Latenz linear mit T_{think} bis zur unteren Schranke T_{exec} , die bei $T_{\text{think}} \geq T_{\text{prep}}$ erreicht wird. Rechts: Latenzgewinn $\Delta = \min(T_{\text{think}}, T_{\text{prep}})$ gemäß Satz 6 – ein stückweise linearer, konkaver Verlauf mit Sättigung bei T_{prep} .

4.6 Robustheit: Idempotenz und Replay-Sicherheit

Definition 11 (Erweiterter Automat mit Idempotenz-Schleifen). Sei $P = (S, \Sigma, \delta, s_0, F)$ ein Prozessautomat. Der *idempotent erweiterte Automat* $P^+ = (S, \Sigma, \delta^+, s_0, F)$ ist definiert durch

$$\delta^+(s, \sigma) = \begin{cases} \delta(s, \sigma), & \text{falls } \delta(s, \sigma) \text{ definiert ist,} \\ s, & \text{falls } \exists s_{\text{prev}} \in S : \delta(s_{\text{prev}}, \sigma) = s \text{ und } \sigma \text{ die zuletzt erfolgreich} \\ & \text{ausgeführte Operation war,} \\ \text{undef,} & \text{sonst.} \end{cases}$$

Das heißt: Wiederholt ein Client genau die Operation, die ihn zuletzt erfolgreich in den aktuellen Zustand s überführt hat (z. B. weil die ursprüngliche Antwort durch einen Netzwerkfehler verloren ging), so verbleibt der Automat in s und liefert erneut die (gecachte) Erfolgsantwort, statt eine Sequenzverletzung zu melden.

Satz 7 (Replay-Sicherheit). Sei P^+ der idempotent erweiterte Automat zu P gemäß Definition 11, und sei $c = (c_1, \dots, c_k)$ vollständig konform bezüglich P mit Lauf $s_0, \dots, s_k$. Sei $\hat{c}$ eine Sequenz, die aus c durch beliebig häufige unmittelbare Wiederholung einzelner Elemente entsteht (d. h. jedes $\hat{c}_j \in \{c_i, c_i\}$ für ein passendes i , wobei die relative Reihenfolge der c_i erhalten bleibt und Wiederholungen stets unmittelbar auf das Original folgen). Dann ist $\hat{c}$ vollständig konform bezüglich P^+ , und der erreichte Endzustand ist identisch zu s_k .

Beweis. Wir zeigen die Aussage durch Induktion über die Position j in $\hat{c}$, mit der Invariante, dass nach Verarbeitung des durch $\hat{c}_1, \dots, \hat{c}_j$ definierten Präfixes der Zustand von P^+ gleich s_i ist, wobei i die Anzahl der *nicht wiederholten* (d. h. „neuen“) Elemente in $\hat{c}_1, \dots, \hat{c}_j$ ist.

Fall „neues Element“: Ist $\hat{c}_j = c_{i+1}$ ein bisher nicht in dieser Position wiederholtes Element, so gilt nach Induktionsannahme $\pi(id) = s_i$ und nach Konformität von c ist $\delta(s_i, c_{i+1}) = s_{i+1}$ definiert; nach Definition 11 gilt $\delta^+ = \delta$ in diesem Fall, also $\pi(id) := s_{i+1}$.

Fall „Wiederholung“: Ist $\hat{c}_j$ eine unmittelbare Wiederholung von $\hat{c}_{j-1} = c_{i+1}$ (der zuletzt erfolgreich ausgeführten Operation, die $\pi(id)$ von s_i nach s_{i+1} überführt hat), so greift der zweite Fall von Definition 11 mit $s = s_{i+1}$, $s_{\text{prev}} = s_i$, $\sigma = c_{i+1}$: $\delta^+(s_{i+1}, c_{i+1}) = s_{i+1}$, also bleibt $\pi(id) = s_{i+1}$ unverändert – die Anzahl der „neuen“ Elemente bleibt $i + 1$, die Invariante bleibt erhalten.

Nach Verarbeitung von $\hat{c}$ entspricht die Anzahl der „neuen“ Elemente genau k (da c vollständig k Elemente hat und jede Wiederholung per Definition keinem neuen Element entspricht), also ist $\pi(id) = s_k$. Da $s_k \in F$ (vollständige Konformität von c), ist auch $\hat{c}$ vollständig konform bezüglich P^+ mit demselben Endzustand. $\square$

Bemerkung 9. Satz 7 zeigt, dass die strikte Sequenzkontrolle aus Definition 6 *nicht* im Widerspruch zu robusten Netzwerkprotokollen steht: Ein Client, der wegen eines Zeitüberschreitung (Timeout) eine erfolgreiche Anfrage erneut sendet, wird durch P^+ nicht fälschlich mit 409 **Conflict** abgewiesen. In der Praxis wird δ^+ üblicherweise mittels eines *Idempotency-Key-Headers* realisiert (vgl. die Diskussion bei Helland [10] zu „at-least-once“-Semantik in verteilten Systemen): Der Server speichert zu jedem $(id, \text{Idempotency-Key})$ -Paar die zuletzt gesendete Antwort und liefert diese bei Wiederholung erneut aus, ohne die Fachlogik erneut auszuführen.

5 Empirische Veranschaulichung

Zur Veranschaulichung der formalen Ergebnisse aus Abschnitt 4 wird die Einführung eines SC-REST-Servers in einem simulierten Szenario betrachtet: Ein Integrations-Team bindet schrittweise einen neuen Client an den Bestellprozess aus Beispiel 1 (Abbildung 1) an.

Die Kombination der Abbildungen 1, 3, 2 und 4 veranschaulicht den vollständigen Wirkungszusammenhang des SC-REST-Patterns: Der Prozessautomat (Abbildung 1) definiert sowohl die zulässigen Sequenzen als auch die Menge $\Sigma(s)$, die im Fehlerfall kommuniziert wird (Satz 1); diese Information ermöglicht serverseitiges Prefetching mit beweisbarem Latenzgewinn (Abbildung 3, Satz 6); für lineare Abschnitte des Prozesses bleibt die Prüfung dabei $\Theta(k)$ -optimal, während permutierbare Abschnitte strukturell auf den optimalen Subgraph Algorithmus zurückgeführt werden (Abbildung 2, Satz 5); und der Gesamtnutzen kumuliert über den Lebenszyklus einer Client-Integration (Abbildung 4).

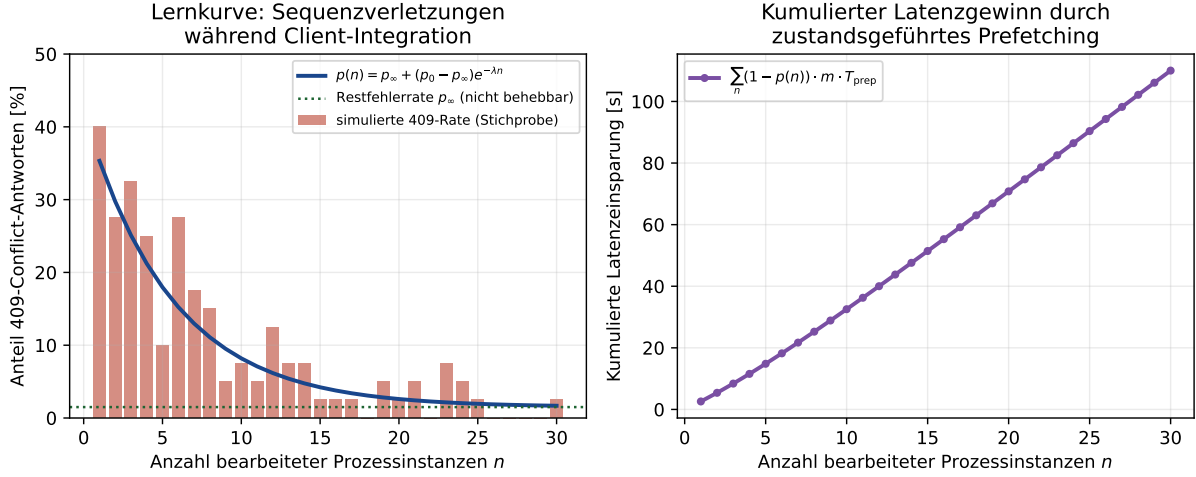


Abbildung 4: Links: Simulierter Anteil von 409 Conflict-Antworten über $n = 1, \dots, 30$ bearbeitete Prozessinstanzen, modelliert als exponentieller Lernprozess $p(n) = p_\infty + (p_0 - p_\infty)e^{-\lambda n}$ mit $p_0 = 0,42$, $p_\infty = 0,015$, $\lambda = 0,18$ (durchgezogene blaue Kurve) und einer binomialverteilten Stichprobe mit $m = 40$ Anfragen je Instanz (rote Balken, Zufallsgenerator-Seed 42). Die anfänglich hohe Fehlerrate sinkt rasch, da Sequenzverletzungen gemäß Satz 1(c) sofort und mit präziser `allowedOperations`-Angabe zurückgemeldet werden – der Client kann sein Verhalten unmittelbar korrigieren. Eine Restfehlerrate $p_\infty > 0$ bleibt bestehen (z. B. durch seltene Edge-Cases oder konkurrierende Prozessinstanzen). Rechts: Kumulierte Latenzeinsparung gemäß $\sum_n (1 - p(n)) \cdot m \cdot T_{\text{prep}}$ mit $T_{\text{prep}} = 100 \text{ ms}$ – jede konform abgeschlossene Anfrage profitiert vom in Satz 6 bewiesenen Latenzgewinn durch Prefetching, sodass sich der Gesamtgewinn über viele Prozessinstanzen linear mit der Trefferquote $1 - p(n)$ aufsummiert.

6 Implementierung

Abschnitt 3 hat bereits die Kernfunktion `handle_request` (Algorithmus 3) eingeführt. Listing 4 zeigt eine vollständigere Referenzimplementierung als Flask-Middleware, die die Übergangsfunktion δ deklarativ definiert, Prefetching gemäß Abschnitt 4.5 anstößt und die Problem-Detail-Antworten aus Abschnitt 3 erzeugt.

Listing 4: Referenzimplementierung eines SC-REST-Prozessautomaten in Flask

```

1 from dataclasses import dataclass, field
2 from typing import Dict, Tuple, Set, List
3 from concurrent.futures import ThreadPoolExecutor
4 from flask import Flask, jsonify, request, abort
5
6 app = Flask(__name__)
7 executor = ThreadPoolExecutor(max_workers=8)
8
9 @dataclass
10 class ProcessAutomaton:
11     states: Set[str]
12     delta: Dict[Tuple[str, str], str]
13     start: str
14     final: Set[str]
15     pi: Dict[str, str] = field(default_factory=dict)

```

```

16
17 def allowed_ops(self, state: str) -> List[str]:
18     return [op for (s, op) in self.delta if s == state]
19
20 def reachable_states(self, state: str) -> Set[str]:
21     return {self.delta[(state, op)]
22             for op in self.allowed_ops(state)}
23
24
25 # Definition 2: Prozessautomat fuer den Bestellprozess
26 order_process = ProcessAutomaton(
27     states={"s0", "s1", "s2", "s3", "s4", "s5"},
28     delta={
29         ("s0", "POST_/cart"): "s1",
30         ("s1", "POST_/cart/items"): "s2",
31         ("s2", "POST_/cart/items"): "s2",
32         ("s2", "POST_/checkout"): "s3",
33         ("s3", "POST_/payment"): "s4",
34         ("s4", "POST_/confirm"): "s5",
35     },
36     start="s0",
37     final={"s5"},
38 )
39
40
41 def prefetch_resources(state: str) -> None:
42     """Definition 9/10: spekulative Vorbereitung fuer alle s' in N(s)."""
43     for target_state in order_process.reachable_states(state):
44         executor.submit(prepare_state_resources, target_state)
45
46
47 def prepare_state_resources(target_state: str) -> None:
48     # z.B. Inventar reservieren, Zahlungsdienstleister vorab
49     # autorisieren, Bestaetigungs-Template vorrendern, ...
50     ...
51
52
53 @app.route("/orders/<order_id>/<path:op_path>", methods=["POST"])
54 def process_step(order_id: str, op_path: str):
55     operation = f"POST_{op_path}"
56     current_state = order_process.pi.get(order_id, order_process.start)
57
58     next_state = order_process.delta.get((current_state, operation))
59
60     if next_state is None:
61         # Satz 1 (Korrektheit, Fehlerlokalisierung):
62         # Zustand bleibt unveraendert, Fehler ist eindeutig lokalisiert
63         return jsonify({
64             "type": "https://api.example.com/probs/sequence-violation",
65             "title": "Ungueltige_Operation_fuer_aktuellen_Prozesszustand",
66             "status": 409,

```

```

67         "instance": f"/orders/{order_id}",
68         "currentState": current_state,
69         "attemptedOperation": operation,
70         "allowedOperations":
71             order_process.allowed_ops(current_state),
72     }), 409
73
74     # Konformer Uebergang (Satz 1b: Completeness)
75     result = execute_business_logic(operation, order_id)
76     order_process.pi[order_id] = next_state
77
78     # Satz 5 (Latenzgewinn): Vorbereitung fuer N(next_state)
79     # waehrend der Bedenkzeit des Clients anstossen
80     prefetch_resources(next_state)
81
82     return jsonify({
83         "orderId": order_id,
84         "state": next_state,
85         "result": result,
86         "_links": hateoas_links(order_process, next_state, order_id),
87     }), 201

```

7 Von SC-REST zu HST: Horizon State Transfer

7.1 Motivation: Das Navigationsdefizit von SC-REST

Das Akronym REST steht für *Representational State Transfer*; namensgebend ist die Beschränkung der *Zustandslosigkeit* der Kommunikation [1]. Präzise gefasst verlangt diese Beschränkung, dass jede Anfrage alle zur Bearbeitung notwendigen Informationen enthält und der Server keinen *Sitzungszustand* (*session state*) führt – also keinen Zustand, der ausschließlich der Kommunikation mit einem bestimmten Client dient und ohne den frühere Anfragen dieses Clients nicht interpretierbar wären. Davon zu unterscheiden ist der *Ressourcenzustand* (in [1] auch als Teil des *application state* verstanden): persistente, über die Ressourcen-URI adressierbare Daten, die unabhängig vom anfragenden Client existieren.

Die Bemerkung zur Zustandslosigkeit in Abschnitt 3 stellt fest, dass SC-REST diese Unterscheidung respektiert: $\pi(id) = s$ ist Ressourcenzustand, kein Sitzungszustand. SC-REST ist insofern bereits REST-konform. Konsequenz zu Ende gedacht zeigt sich jedoch eine zweite, bislang unadressierte Asymmetrie zwischen Server und Client – das *Navigationsdefizit*.

Formal kennt der Server zu jedem Zeitpunkt nicht nur den aktuellen Zustand $\pi(id) = s$, sondern den *gesamten* Prozessautomaten $P = (S, \Sigma, \delta, s_0, F)$ – insbesondere für jeden erreichbaren Zustand $s' \in N(s)$ wiederum $\Sigma(s')$, $N(s')$ usw. Gemäß Definition 6 übermittelt ein SC-REST-Server in seiner Antwort jedoch nur $\Sigma(\delta(s, \sigma))$ – also Informationen über genau *einen* Schritt voraus. Ein Client, der $k > 1$ Schritte planen möchte (etwa um zu entscheiden, ob ein mehrstufiger Vorgang überhaupt zulässig ist, oder um – wie in Abschnitt 4.5 diskutiert – Ressourcen für mehrere Schritte vorzubereiten), muss dafür k aufeinanderfolgende Anfragen stellen, *obwohl* die hierfür nötige Information (die Ein-

schränkung von δ auf den relevanten Ausschnitt) bereits zum Zeitpunkt der ersten Antwort vollständig und deterministisch beim Server vorliegt.

Bemerkung 10. Das Navigationsdefizit ist kein Verstoß gegen die Zustandslosigkeit im Sinne von [1] – die einzelne Anfrage bleibt selbstbeschreibend, und der Server speichert keinen client-spezifischen Sitzungszustand. Es handelt sich jedoch um eine *Unvollständigkeit der übertragenen Repräsentation* gegenüber dem, was REST mit der Beschränkung der *Selbstbeschreibungsfähigkeit* und insbesondere mit HATEOAS anstrebt: Eine Repräsentation soll dem Client ermöglichen, ohne über die Anwendung hinausgehendes Wissen fortzufahren. SC-REST realisiert HATEOAS nur mit *Horizont* $h = 1$ – der folgende Abschnitt führt den Horizont h als expliziten Parameter ein und verallgemeinert SC-REST entsprechend.

Ziel dieses Abschnitts ist die formale Definition eines Patterns, das dieses Defizit behebt, ohne die Zustandslosigkeit (Abschnitt 7.3) einzuschränken. Wir nennen dieses Pattern *Horizon State Transfer (HST)*, da – wie sich zeigen wird – die zusätzlich übertragene Information genau der von s aus in höchstens h Schritten erreichbare Teilgraph des Prozessautomaten ist (formal behandelt in [12]), also der *Navigationshorizont* der Tiefe h . Der Name ist bewusst in Analogie zu *Representational State Transfer* gewählt: Während REST in jeder Antwort eine *Repräsentation* des aktuellen Zustands überträgt, überträgt HST zusätzlich den vollständigen *Horizont* G_s^h des Zustandsraums – den lokal relevanten Ausschnitt des Navigationswissens, das der Server ohnehin vollständig besitzt, begrenzt durch den Parameter h .

7.2 Das HST-Pattern: Formale Definition

Definition 12 (*h-Schritt-Horizont-Subgraph*). Sei $P = (S, \Sigma, \delta, s_0, F)$ ein Prozessautomat, $s \in S$ und $h \in \mathbb{N}_0$. Wir fassen P als gerichteten, mit Σ beschrifteten Graphen $(S, \rightarrow)$ auf, mit $s_1 \xrightarrow{\sigma} s_2 \Leftrightarrow \delta(s_1, \sigma) = s_2$. Für $(c_1, \dots, c_k) \in \Sigma^k$ sei $\delta^*(s, (c_1, \dots, c_k))$ der gemäß Definition 3 erreichte Zustand nach Ausführung von $(c_1, \dots, c_k)$ ab s , mit $\delta^*(s, ()) = s$. Der *h-Schritt-Horizont-Subgraph* $G_s^h = (V_s^h, E_s^h)$ ist definiert durch

$$V_s^h := \{ s' \in S \mid \exists k \leq h, \exists c \in \Sigma^k : s' = \delta^*(s, c) \}$$

und, für $h \geq 1$,

$$E_s^h := \{ (s_1, \sigma, s_2) \mid s_1 \in V_s^{h-1}, \sigma \in \Sigma(s_1), s_2 = \delta(s_1, \sigma) \},$$

mit $E_s^0 := \emptyset$. Jeder Knoten $s' \in V_s^h \setminus V_s^{h-1}$ (für $h \geq 1$) heißt *Randknoten* von G_s^h ; die Menge der Randknoten wird mit ∂G_s^h bezeichnet.

Anschaulich ist G_s^h der induzierte Teilgraph, der durch eine Breitensuche (BFS) der Tiefe h ab s im Prozessautomaten entsteht – einschließlich aller Kanten, deren Startknoten in höchstens $h - 1$ Schritten ab s erreichbar ist.

Bemerkung 11. $G_s^0 = (\{s\}, \emptyset)$, und G_s^1 entspricht genau der in Definition 6 übertragenen Information: $V_s^1 = \{s\} \cup N(s)$ mit den Kanten $\{(s, \sigma, \delta(s, \sigma)) \mid \sigma \in \Sigma(s)\}$. **SC-REST ist somit der Spezialfall $h = 1$ von HST.**

Definition 13 (*HST-Repräsentation*). Sei (id, s) eine Prozessinstanz gemäß Definition 5 und $h \in \mathbb{N}$ ein (server- oder anwendungsseitig konfigurierter) *Horizontparameter*. Die *HST-Repräsentation* der Prozessinstanz ist das Tupel

$$R(id, s, h) := (\rho(id, s), s, G_s^h),$$

wobei $\rho(id, s)$ die fachliche Ressourcenrepräsentation (Nutzdaten der Prozessinstanz) bezeichnet und G_s^h gemäß Definition 12 als annotierter Graph übertragen wird: Jeder Knoten $s' \in V_s^h$ wird auf eine URI (bzw. einen URI-Template-Parameter) abgebildet, jede Kante $(s_1, \sigma, s_2) \in E_s^h$ auf einen HATEOAS-Link mit Methode, Pfad und annotiertem Zielzustand s_2 .

Definition 14 (HST-Operationssemantik). Sei $P = (S, \Sigma, \delta, s_0, F)$ ein Prozessautomat, $\pi(id) = s$ der aktuelle Zustand einer Prozessinstanz und $h \in \mathbb{N}$ der Horizontparameter. Auf eine eingehende Anfrage $\sigma \in \Sigma$ reagiert ein *HST-Server* wie folgt:

1. **Falls $\delta(s, \sigma)$ definiert ist:** Wie in Definition 6 (Fachlogik ausführen, $\pi(id) := \delta(s, \sigma)$), jedoch antwortet der Server mit der HST-Repräsentation $R(id, \delta(s, \sigma), h)$ statt nur mit $\Sigma(\delta(s, \sigma))$.
2. **Falls $\delta(s, \sigma)$ undefiniert ist:** Wie in Definition 6 (409 Conflict mit Problem-Detail-Objekt, $\pi(id)$ bleibt unverändert); zusätzlich enthält die Antwort weiterhin G_s^h (unverändert, da $\pi(id)$ unverändert bleibt), sodass der Client den Sequenzfehler unmittelbar im Kontext seines bereits bekannten Horizont-Subgraphen einordnen kann.

Beispiel 2 (Bestellprozess mit Horizont $h = 2$). Für den Prozessautomaten aus Beispiel 1 und $s = s_1$ (Zustand nach `POST /cart`) ergibt sich für $h = 2$:

$$V_{s_1}^2 = \{s_1, s_2, s_3\},$$

$$E_{s_1}^2 = \{ (s_1, \text{POST /cart/items}, s_2), (s_2, \text{POST /cart/items}, s_2), \\ (s_2, \text{POST /checkout}, s_3) \}.$$

Ein Client, der $G_{s_1}^2$ erhält, kann *ohne weitere Serveranfrage* feststellen, dass `POST /checkout` aus s_1 direkt nicht zulässig ist, jedoch nach genau einem `POST /cart/items` zulässig wird – eine Information, die in der SC-REST-Repräsentation ($h = 1$, nur $\Sigma(s_1) = \{\text{POST /cart/items}\}$) nicht enthalten ist.

7.3 Zustandslosigkeit von HST

Satz 8 (Zustandslosigkeit von HST). Für jeden Horizont $h \in \mathbb{N}$ erfüllt ein HST-Server gemäß Definition 14 die Zustandslosigkeits-Beschränkung nach [1]: Der Server führt keinen client-spezifischen Sitzungszustand, und jede Antwort ist eine deterministische Funktion ausschließlich von (a) dem Prozessautomaten P und (b) dem Ressourcenzustand $\pi(id) = s$.

Beweis. Wir zeigen, dass $R(id, s, h)$ gemäß Definition 13 vollständig durch (P, id, s, h) bestimmt ist, unabhängig davon, welcher Client die Anfrage stellt oder welche Anfragen zuvor gestellt wurden.

(1) $\rho(id, s)$ ist nach Definition 5 die fachliche Repräsentation der Ressource id im Zustand s – eine Funktion von (id, s) , unabhängig vom anfragenden Client.

(2) G_s^h ist nach Definition 12 eine Funktion von (P, s, h) : Die Mengen V_s^h und E_s^h werden ausschließlich mittels δ^* – also iterierter Anwendung von δ , einer Komponente von P – aus s berechnet. In diese Berechnung geht insbesondere keine Information über die Identität des anfragenden Clients, über dessen vorherige Anfragen oder über andere Prozessinstanzen ein.

(3) h ist nach Voraussetzung ein server- oder anwendungsseitig (nicht client-seitungsseitig) konfigurierter Parameter – etwa eine Konstante des Prozessautomaten P oder ein in der aktuellen Anfrage selbst übermittelter Parameter (z. B. Query-Parameter `?horizon=2`); in beiden Fällen ist h Teil der Eingabe der aktuellen Anfrage und keine gespeicherte Information.

Damit ist $R(id, s, h) = (\rho(id, s), s, G_s^h)$ eine Funktion von (P, id, s, h) , wobei P statisch (bzw. unabhängig versioniert, siehe Abschnitt 7.7) und $\pi(id) = s$ Ressourcenzustand im Sinne der Bemerkung zur Zustandslosigkeit in Abschnitt 3 ist. Insbesondere benötigt der Server zur Beantwortung *keinen* zusätzlichen Speicher pro Client oder pro Sitzung – die einzige persistente, über die Anfrage hinaus gespeicherte Information ist $\pi(id)$, welche bereits für SC-REST ($h = 1$) als Ressourcenzustand klassifiziert wurde. Da diese Klassifikation nicht von h abhängt, gilt sie für beliebiges $h \in \mathbb{N}$, also auch für HST. $\square$

Bemerkung 12. Satz 8 zeigt, dass die Erhöhung des Horizonts h *zustandslosigkeitsneutral* ist: h beeinflusst ausschließlich die Größe der Antwort (Abschnitt 7.4), nicht deren Charakter als zustandslose, selbstbeschreibende Nachricht. Der vermeintliche Zielkonflikt zwischen „mehr Navigationsinformation für den Client“ und „REST-Konformität“ – der die in Bemerkung 10 beschriebene Beschränkung auf $h = 1$ in der Praxis oft motiviert – existiert formal nicht; lediglich der in Abschnitt 7.6 formalisierte Kosten-Nutzen-Tradeoff begrenzt h auf einen endlichen, optimalen Wert h^* .

7.4 Größe des Horizont-Subgraphen

Lemma 2 (Größenschranke für G_s^h). Sei $P = (S, \Sigma, \delta, s_0, F)$ ein Prozessautomat und $d := \max_{s' \in S} d(s')$ der maximale Verzweigungsgrad (Definition 2). Für jeden Zustand $s \in S$ und jeden Horizont $h \in \mathbb{N}_0$ gilt

$$|V_s^h| \leq \sum_{i=0}^h d^i = \begin{cases} h+1, & d = 1, \\ \frac{d^{h+1} - 1}{d - 1}, & d \neq 1. \end{cases}$$

Gleichheit gilt genau dann, wenn der von s aus erreichbare Teil von P bis zur Tiefe h ein Baum mit konstantem Verzweigungsgrad d ist, d. h. wenn δ^* auf Pfaden der Länge $\leq h$ ab s injektiv ist (keine zwei verschiedenen solchen Pfade führen zum selben Zustand) und jeder Zustand auf Tiefe $< h$ genau d ausgehende Operationen besitzt.

Beweis. Wir zerlegen V_s^h in Schichten $L_0, \dots, L_h$ mit $L_0 := \{s\}$ und, für $i \geq 1$, $L_i := V_s^i \setminus V_s^{i-1}$ (mit $V_s^{-1} := \emptyset$). Wir zeigen $|L_i| \leq d^i$ für alle $i \leq h$ durch Induktion über i .

Induktionsanfang ($i = 0$): $L_0 = \{s\}$, also $|L_0| = 1 = d^0$.

Induktionsschritt: Sei $|L_{i-1}| \leq d^{i-1}$ gezeigt. Jeder Knoten $s' \in L_i$ ist nach Definition 12 von der Form $s' = \delta(s'', \sigma)$ für ein $s'' \in L_{i-1}$ und $\sigma \in \Sigma(s'')$ – denn $s' \in V_s^i \setminus V_s^{i-1}$ bedeutet, dass der kürzeste Pfad von s nach s' genau Länge i hat, also über einen Knoten s'' auf Tiefe $i - 1$ verläuft (andernfalls wäre s' bereits über einen kürzeren Pfad in V_s^{i-1} enthalten). Da $d(s'') \leq d$ für alle $s'' \in S$ (Definition von d als Maximum), gibt es zu jedem $s'' \in L_{i-1}$ höchstens d Operationen $\sigma \in \Sigma(s'')$ und damit höchstens d Kandidaten für Nachfolgeknoten. Da $L_i \subseteq \bigcup_{s'' \in L_{i-1}} \{\delta(s'', \sigma) \mid \sigma \in \Sigma(s'')\}$, folgt

$$|L_i| \leq |L_{i-1}| \cdot d \leq d^{i-1} \cdot d = d^i.$$

Damit folgt $|V_s^h| = \sum_{i=0}^h |L_i| \leq \sum_{i=0}^h d^i$.

Gleichheitsfall: Gleichheit $|L_i| = d \cdot |L_{i-1}|$ für alle $i \leq h$ gilt genau dann, wenn (a) jeder Knoten $s'' \in L_{i-1}$ *genau* d ausgehende Operationen besitzt (der maximale Verzweigungsgrad wird an jedem dieser Knoten tatsächlich erreicht) *und* (b) die resultierenden $d \cdot |L_{i-1}|$ Nachfolgeknoten paarweise verschieden sind und nicht bereits in V_s^{i-1} liegen. Bedingungen (a) und (b) für alle $i \leq h$ sind äquivalent dazu, dass δ^* auf Pfaden der Länge $\leq h$ ab s injektiv ist und jeder Knoten auf Tiefe $< h$ Verzweigungsgrad genau d hat – d. h. der von s aus erreichbare Teilgraph bis Tiefe h ist ein d -ärer Baum. $\square$

Bemerkung 13. Lemma 2 zeigt, dass $|V_s^h|$ für $d \geq 2$ *exponentiell* in h wächst. Für die in Abschnitt 4.4 betrachteten Sammelzustände mit r permutierbaren optionalen Teilschritten ist dies kein Widerspruch zu Bemerkung 7: Dort wird $|S| \geq 2^r$ als *untere* Schranke für die naive Kodierung des *gesamten* Automaten gezeigt, während Lemma 2 eine *obere* Schranke für den *lokal pro Anfrage übertragenen* Teilgraphen G_s^h liefert – für $h < r$ ist $|V_s^h|$ um Größenordnungen kleiner als $|S|$. Abbildung 6 illustriert das Wachstum von $|V_s^h|$ für $d \in \{1, 2, 3\}$.

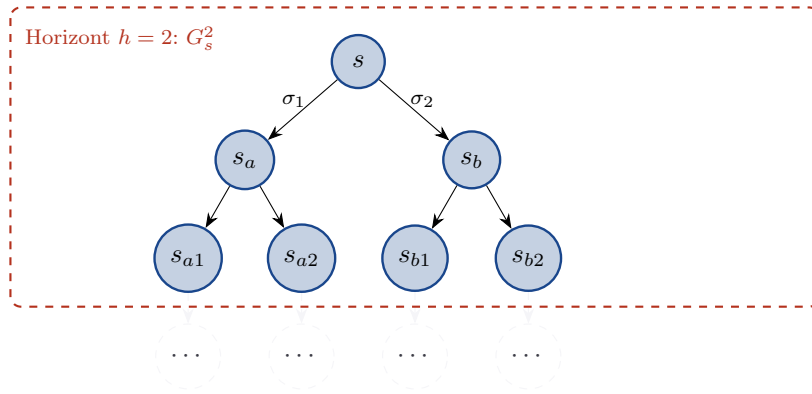


Abbildung 5: Schematische Darstellung des Horizont-Subgraphen G_s^2 (blau, innerhalb der gestrichelten roten Umrandung) für einen Prozessautomaten mit Verzweigungsgrad $d = 2$ ab dem aktuellen Zustand s . Gemäß Lemma 2 enthält G_s^2 höchstens $\sum_{i=0}^2 2^i = 1 + 2 + 4 = 7$ Knoten – hier mit Gleichheit, da der dargestellte Ausschnitt ein binärer Baum ist. Die hellgrau gestrichelten Knoten ($\dots$) liegen auf Tiefe 3, also außerhalb des Horizonts, und sind dem Client ohne weitere Anfrage nicht bekannt: Erst nach einem Zustandsübergang innerhalb des Horizonts (z. B. $s \xrightarrow{\sigma_1} s_a$) liefert der Server den *aktualisierten* Subgraphen $G_{s_a}^2$, dessen Randknoten dann diese zuvor verborgenen Zustände enthält. Der Horizont „wandert“ also mit der Prozessinstanz mit, ohne dass der Server jemals den vollständigen Automaten P in einer einzelnen Antwort übertragen muss.

7.5 Lokale Vollständigkeit und Planungshorizont

Satz 9 (Lokale Vollständigkeit von G_s^h). Sei (id, s) eine Prozessinstanz und G_s^h der zugehörige Horizont-Subgraph gemäß Definition 12. Für jede Aufrufsequenz $c = (c_1, \dots, c_k)$ mit $k \leq h$ gilt: Die Frage, ob c ab s (partiell) konform ist (Definition 4), und – falls ja – welcher Zustand $\delta^*(s, c)$ erreicht wird, ist *vollständig aus G_s^h entscheidbar*, ohne weitere Anfrage an den Server.

Beweis. Wir zeigen zunächst die monotone Inklusion $V_s^i \subseteq V_s^j$ für $i \leq j$: Jeder in höchstens i Schritten ab s erreichbare Zustand ist insbesondere auch in höchstens $j \geq i$ Schritten

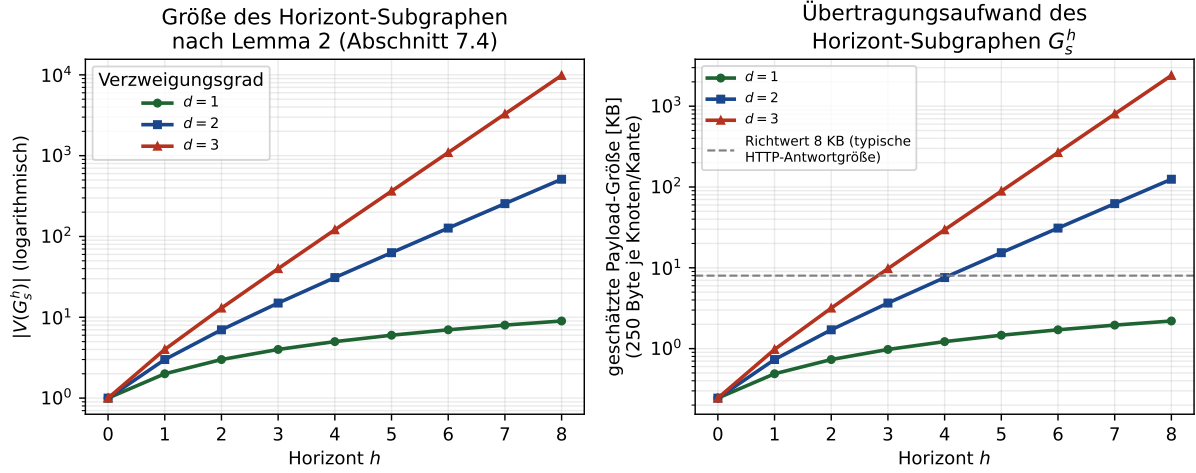


Abbildung 6: Links: Anzahl der Knoten $|V_s^h|$ des Horizont-Subgraphen (logarithmische Skala) in Abhängigkeit vom Horizont h , für maximale Verzweigungsgrade $d = 1$ (lineare Prozessabschnitte ohne Verzweigung, z. B. der Hauptpfad in Beispiel 1), $d = 2$ und $d = 3$, gemäß der Schranke aus Lemma 2 (mit angenommener Gleichheit als Worst-Case). Für $d = 1$ wächst $|V_s^h|$ nur linear ($h + 1$), für $d \geq 2$ exponentiell – bereits für $d = 3$ und $h = 8$ ergeben sich knapp 10^4 Knoten. Rechts: geschätzte Übertragungsgröße der HST-Repräsentation $R(id, s, h)$ in Kilobyte, unter der konservativen Annahme von ca. 250 Byte pro Knoten bzw. Kante (URI, Methode, Zustandskennung, JSON-Strukturoverhead). Die gestrichelte Linie markiert 8 KB als groben Richtwert für eine „leichtgewichtige“ HTTP-Antwort. Für $d = 2$ wird dieser Wert bereits bei $h \approx 4$, für $d = 3$ bei $h \approx 3$ überschritten – dies motiviert den in Abschnitt 7.6 formalisierten kostenoptimalen Horizont h^* , der für praxisrelevante Parameter deutlich kleiner ist als die in Abschnitt 4.4 betrachteten Werte $r \leq 20$.

erreichbar (man kann dieselbe Sequenz verwenden), also folgt die Inklusion direkt aus Definition 12.

Wir zeigen die Behauptung nun durch Induktion über $k \leq h$.

Induktionsanfang ($k = 0$): $c = ()$ ist nach Definition 4 trivial konform mit $\delta^*(s, ()) = s \in V_s^h$ für jedes $h \geq 0$.

Induktionsschritt: Sei die Aussage für $k - 1 < h$ Schritte gezeigt, d. h. es existiert (unter der Annahme, dass $(c_1, \dots, c_{k-1})$ konform ist) ein Zustand $s_{k-1} := \delta^*(s, (c_1, \dots, c_{k-1})) \in V_s^{k-1}$, und dieser Zustand sowie die Konformität von $(c_1, \dots, c_{k-1})$ sind aus G_s^h bestimmbar. Wegen $k - 1 \leq h - 1$ und der oben gezeigten Monotonie gilt $s_{k-1} \in V_s^{h-1}$.

Zu prüfen ist, ob $\delta(s_{k-1}, c_k)$ definiert ist. Da $s_{k-1} \in V_s^{h-1}$, ist nach Definition 12 jede Operation $\sigma \in \Sigma(s_{k-1})$ Quelle einer Kante $(s_{k-1}, \sigma, \delta(s_{k-1}, \sigma)) \in E_s^h$ – die Kantenmenge E_s^h enthält per Definition alle ausgehenden Kanten jedes Knotens aus V_s^{h-1} , unabhängig davon, ob deren Zielknoten neu (Randknoten von G_s^h) oder bereits in V_s^{h-1} enthalten ist. Somit gilt:

- $\delta(s_{k-1}, c_k)$ ist definiert $\iff$ es existiert eine Kante $(s_{k-1}, c_k, s_k) \in E_s^h$ für ein $s_k \in V_s^h$ – diese Bedingung ist direkt aus der (endlichen) Kantenmenge E_s^h ablesbar.
- Ist $\delta(s_{k-1}, c_k)$ definiert, so ist der gemäß Definition 2 eindeutige Zielknoten s_k dieser Kante genau $\delta^*(s, (c_1, \dots, c_k))$, und $s_k \in V_s^k \subseteq V_s^h$ (wieder nach Monotonie, $k \leq h$).
- Ist $\delta(s_{k-1}, c_k)$ undefiniert, so ist c ab Schritt k nicht konform – auch diese Information

ist aus E_s^h ablesbar (Abwesenheit einer entsprechenden Kante mit Quelle s_{k-1} und Beschriftung c_k).

In beiden Fällen ist die Konformität von c bis Schritt k und ggf. der erreichte Zustand $\delta^*(s, (c_1, \dots, c_k))$ vollständig aus G_s^h bestimmbar. Per Induktion folgt die Behauptung für alle $k \leq h$. $\square$

Bemerkung 14. Satz 9 hat eine direkte Konsequenz für die in Abschnitt 4.4 behandelten Sammelzustände: Ist s_c ein Sammelzustand mit r permutierbaren optionalen Teilschritten (Definition 8) und wählt der Server den Horizont $h \geq r$, so enthält $G_{s_c}^h$ – mit $d = 2$ an jedem Zwischenzustand (Operation σ_j ausführen oder nicht) und damit nach Lemma 2 mit $|V_{s_c}^h| \leq \sum_{i=0}^r 2^i = 2^{r+1} - 1$ Knoten – den *vollständigen* Entscheidungsbaum aller $r!$ möglichen Ausführungsreihenfolgen. Ein Client kann damit für eine *beliebige*, lokal geplante Reihenfolge c' der r Teilschritte mittels Satz 9 *vorab und lokal* prüfen, ob c' konform ist – und kann zusätzlich, sofern c' als Beobachtungsfolge im Sinne von Definition 9 interpretiert wird, den Subgraph Algorithmus aus [12] *client-seitig* auf den in $G_{s_c}^h$ enthaltenen Signaturen ausführen (Satz 5(b), $\Theta(r^3)$). Die Konformitätsprüfung für Sammelzustände wird damit *vollständig dezentralisiert*: Der Server muss sie nur noch im Fehlerfall (echte Sequenzverletzung, etwa durch einen vom Plan abweichenden Client) erneut durchführen, vgl. Definition 14(2).

7.6 Kostenoptimaler Horizont

Definition 15 (Kostenmodell für den Horizont). Sei $\alpha > 0$ die marginale Übertragungskosten pro Knoten/Kante des Horizont-Subgraphen (z.B. in normierten Bandbreiten- oder Byte-Einheiten) und $\beta > 0$ die Kosten eines zusätzlichen Anfrage-Antwort-Zyklus (*Round-Trip*), der anfällt, wenn der Client eine Operation ausführen möchte, deren Zielzustand außerhalb von G_s^h liegt (*Horizontüberschreitung*). Wir modellieren die Anzahl der vom Client über den Horizont h hinaus benötigten Schritte als eine Zufallsvariable mit geometrisch fallender Überschreitungswahrscheinlichkeit: Mit *Fortsetzungswahrscheinlichkeit* $p \in (0, 1)$ sei $\Pr[\text{Client überschreitet Horizont } h] = p^{h+1}$, und im Überschreitungsfall seien im Erwartungswert $\frac{1}{1-p}$ zusätzliche Round-Trips erforderlich (geometrische Reihe über weitere mögliche Überschreitungen). Die *erwarteten Gesamtkosten* für Horizont $h \in \mathbb{N}_0$ sind dann

$$C(h) := \alpha \cdot |V_s^h| + \beta \cdot \frac{p^{h+1}}{1-p} = \alpha \sum_{i=0}^h d^i + \beta \cdot \frac{p^{h+1}}{1-p},$$

wobei $|V_s^h| = \sum_{i=0}^h d^i$ gemäß Lemma 2 (Gleichheitsfall, als Worst-Case-Schätzung des Verzweigungsgrads $d \geq 1$) verwendet wird.

Satz 10 (Existenz und Charakterisierung des optimalen Horizonts). Sei $C(h)$ wie in Definition 15, mit $d \geq 1$, $p \in (0, 1)$, $\alpha, \beta > 0$. Dann gilt:

- (a) Die Differenzfunktion $\Delta C(h) := C(h+1) - C(h) = \alpha d^{h+1} - \beta p^{h+1}$ ist streng monoton wachsend in $h \in \mathbb{N}_0$.
- (b) C ist *unimodal*: Es existiert ein eindeutiges $h^* \in \mathbb{N}_0$, sodass $C(0) > C(1) > \dots > C(h^*) \leq C(h^* + 1) < C(h^* + 2) < \dots$ (mit der Konvention, dass die linke Kette leer ist, falls bereits $\Delta C(0) \geq 0$). Insbesondere gilt $C(h^*) = \min_{h \in \mathbb{N}_0} C(h)$.

(c) Es gilt die geschlossene Form

$$h^* = \max\left(0, \left\lceil \log_{d/p}\left(\frac{\beta}{\alpha}\right) \right\rceil - 1\right).$$

Beweis. (a) Da $d \geq 1$ und $0 < p < 1$, gilt $d/p > 1$ (im Fall $d = 1$ ist $d/p = 1/p > 1$ wegen $p < 1$; im Fall $d > 1$ erst recht). Für die zweite Differenz gilt

$$\Delta C(h+1) - \Delta C(h) = (\alpha d^{h+2} - \beta p^{h+2}) - (\alpha d^{h+1} - \beta p^{h+1}) = \alpha d^{h+1}(d-1) + \beta p^{h+1}(1-p).$$

Da $d \geq 1$ (also $d-1 \geq 0$), $0 < p < 1$ (also $1-p > 0$) und $\alpha, \beta > 0$, ist der zweite Summand $\beta p^{h+1}(1-p)$ stets strikt positiv und der erste Summand nicht-negativ. Also $\Delta C(h+1) - \Delta C(h) > 0$ für alle $h \geq 0$, d. h. ΔC ist streng monoton wachsend.

(b) Für $h \rightarrow \infty$ gilt $\Delta C(h) = \alpha d^{h+1} - \beta p^{h+1}$. Da $p^{h+1} \rightarrow 0$ und $d^{h+1} \geq 1$ für alle $h \geq 0$ (mit $d^{h+1} \rightarrow \infty$ falls $d > 1$ bzw. $d^{h+1} = 1$ konstant falls $d = 1$), folgt $\Delta C(h) \rightarrow \alpha \cdot \lim_{h \rightarrow \infty} d^{h+1} \geq \alpha > 0$ für $h \rightarrow \infty$ – insbesondere wird $\Delta C(h)$ für hinreichend großes h positiv. Da ΔC nach (a) streng monoton wachsend ist und für große h positiv wird, gibt es höchstens einen Vorzeichenwechsel von „ < 0 “ zu „ ≥ 0 “. Sei

$$h^* := \min\{h \in \mathbb{N}_0 \mid \Delta C(h) \geq 0\}$$

(wohldefiniert, da die Menge nichtleer ist nach obigem Grenzwertargument). Für $h < h^*$ gilt $\Delta C(h) < 0$ (Minimalität von h^* und Monotonie von ΔC schließen $\Delta C(h) \geq 0$ für $h < h^*$ aus), also $C(h+1) < C(h)$: C ist auf $\{0, \dots, h^*\}$ streng fallend. Für $h \geq h^*$ gilt $\Delta C(h) \geq 0$ wegen Monotonie von ΔC und $\Delta C(h^*) \geq 0$, also $C(h+1) \geq C(h)$, mit strikter Ungleichung für $h > h^*$ (da $\Delta C(h) > \Delta C(h^*) \geq 0$ für $h > h^*$ nach strikter Monotonie). Also ist C auf $\{h^*, h^*+1, \dots\}$ streng wachsend. Damit ist C unimodal mit eindeutigem Minimum bei h^* .

(c) Die definierende Bedingung $\Delta C(h) \geq 0$ ist äquivalent zu

$$\alpha d^{h+1} \geq \beta p^{h+1} \iff \left(\frac{d}{p}\right)^{h+1} \geq \frac{\beta}{\alpha} \iff (h+1) \log \frac{d}{p} \geq \log \frac{\beta}{\alpha} \iff h+1 \geq \log_{d/p} \frac{\beta}{\alpha},$$

wobei die letzte Äquivalenz gilt, da $\log(d/p) > 0$ nach (a) und das Logarithmieren einer Ungleichung mit positiver Basis > 1 die Richtung erhält. Der kleinste ganzzahlige Wert $h \geq 0$ mit $h+1 \geq \log_{d/p}(\beta/\alpha)$ ist

$$h = \max\left(0, \left\lceil \log_{d/p}\left(\frac{\beta}{\alpha}\right) \right\rceil - 1\right)$$

(die Maximumsbildung mit 0 deckt den Fall ab, dass bereits $\Delta C(0) \geq 0$ gilt, also $\log_{d/p}(\beta/\alpha) \leq 1$). Nach Definition von h^* in (b) ist dies genau h^* . $\square$

Bemerkung 15. Satz 10 bestätigt formal die in Bemerkung 12 aufgeworfene Beobachtung: Der optimale Horizont h^* hängt *ausschließlich* vom Kostenverhältnis β/α (Round-Trip- vs. Knotenkosten) und vom Verhältnis d/p (Verzweigungsgrad vs. Fortsetzungswahrscheinlichkeit) ab, *nicht* jedoch von einer prinzipiellen REST-Inkompatibilität (Satz 8). Für hochlatente Verbindungen (großes β , z. B. mobile Clients) oder Prozesse mit hoher Fortsetzungswahrscheinlichkeit p (lange, vorhersehbare Sequenzen, etwa Sammelzustände gemäß Bemerkung 14) wächst h^* . Abbildung 7 (rechts) zeigt diese Abhängigkeit als

Stufenfunktion, da h^* ganzzahlig ist. Insbesondere ist für Sammelzustände mit r permutierbaren Teilschritten der Horizont $h^* \geq r$ (vollständige Dezentralisierung gemäß Bemerkung 14) gemäß (c) genau dann optimal, wenn $\beta/\alpha \geq (d/p)^r$ – für latenzkritische Anwendungen (großes β/α) also bereits für moderate r erfüllt, während für latenzunkritische Anwendungen (kleines β/α) ein kleinerer Horizont $h^* < r$ kosteneffizienter ist und die Sammelzustands-Konformität partiell serverseitig verbleibt.

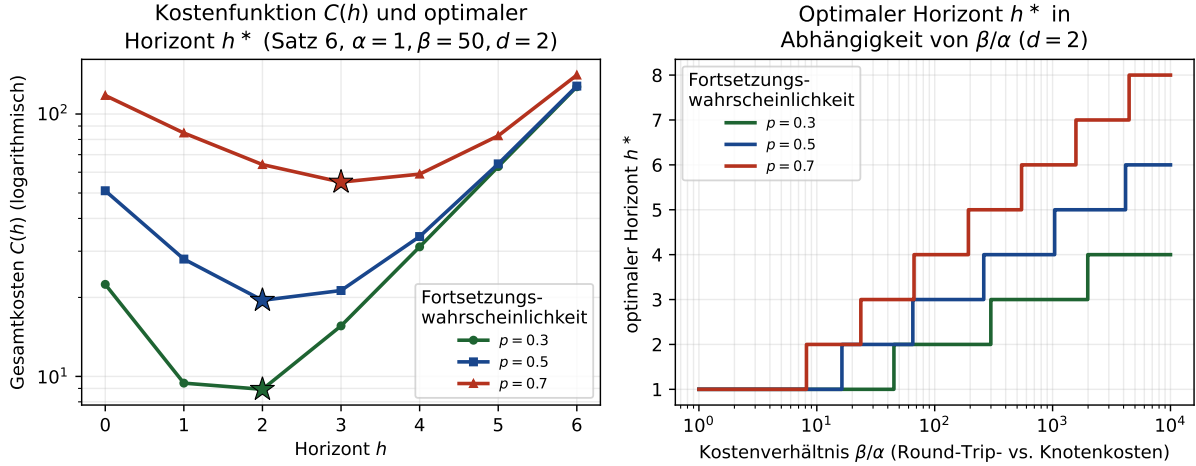


Abbildung 7: Links: Gesamtkosten $C(h)$ (logarithmische Skala) gemäß Definition 15 für $\alpha = 1, \beta = 50, d = 2$ und Fortsetzungswahrscheinlichkeiten $p \in \{0.3, 0.5, 0.7\}$. Die Sterne markieren das nach Satz 10 eindeutige Minimum h^* : Für $p = 0.3$ und $p = 0.5$ liegt es bei $h^* = 2$, für $p = 0.7$ bei $h^* = 3$ – in Übereinstimmung mit der geschlossenen Form aus (c), denn für $p = 0.7$ gilt $\log_{2/0.7}(50) \approx 3.83$, also $h^* = \lceil 3.83 \rceil - 1 = 3$. Rechts: optimaler Horizont h^* als Funktion des Kostenverhältnisses β/α (logarithmische Abszisse) für $d = 2$ und dieselben p -Werte, als Stufenfunktion gemäß (c). Größere β/α (teurere Round-Trips relativ zur Payload-Größe, z. B. bei hoher Netzwerklatenz) und größere p (Prozesse mit tendenziell langen Aufrufsequenzen) führen zu größerem h^* ; die Stufen markieren die in (c) bewiesenen Schwellenwerte $\beta/\alpha = (d/p)^{h+1}$.

7.7 Versionierung und Cache-Kohärenz via Signaturen

Definition 16 (Signatur des Horizont-Subgraphen). Sei $G_s^h = (V_s^h, E_s^h)$ ein Horizont-Subgraph mit einer fest gewählten, kanonischen Aufzählung $V_s^h = \{v_0, \dots, v_{m-1}\}$ (z. B. in BFS-Reihenfolge gemäß Definition 12, mit $v_0 = s$) und sei $\text{idx} : \Sigma \rightarrow \{0, \dots, |\Sigma| - 1\}$ eine feste Indizierung des Operationsalphabets. Die *Signatur* von G_s^h ist

$$\text{Sig}(G_s^h) := \sum_{(v_i, \sigma, v_j) \in E_s^h} \left(i + j \cdot m + \text{idx}(\sigma) \cdot m^2 \right) \cdot (2|\Sigma| m^2)^{\text{rang}(v_i, \sigma, v_j)},$$

wobei $\text{rang}(\cdot)$ eine feste Aufzählung der Kanten $0, 1, 2, \dots$ bezeichnet. Diese Konstruktion überträgt das Prinzip der Spalten-Signaturen $\sigma_j = \sum_i A_{ij} 2^i + j \cdot 2^n$ aus [12] – dort für Adjazenzmatrizen von Aufrufsequenzen definiert – auf die (Knoten-, Kanten-, Beschriftungs-)Tripel von G_s^h : Jede Kante wird auf einen Wert im Bereich $[0, 2|\Sigma|m^2]$ abgebildet und die Kanten werden, analog zur stellenwertigen Kodierung in [12], mit unterschiedlichen „Stellen“ (Potenzen von $2|\Sigma|m^2$) gewichtet, sodass Sig auf der Menge aller Graphen

mit höchstens m Knoten über dem Operationsalphabet Σ injektiv ist (Übertragung des Eindeutigkeitslemmas aus [12] auf die hier verwendete stellenwertige Kodierung).

Bemerkung 16 (Cache-Kohärenz für Navigationsgraphen). Ein Client speichert $(G_s^h, \text{Sig}(G_s^h))$ lokal. Wird der Prozessautomat P – etwa im Rahmen einer fachlichen Prozessänderung – zu P' aktualisiert, sodass sich δ in einem für s relevanten Bereich ändert, so unterscheidet sich der neue Horizont-Subgraph $(G')_s^h$ von G_s^h in mindestens einer Kante. Da Sig nach Definition 16 injektiv ist, folgt $\text{Sig}((G')_s^h) \neq \text{Sig}(G_s^h)$. Ein HST-Server kann die aktuelle Signatur in einem ETag-Header der HST-Repräsentation übermitteln; ein Client kann damit in $O(1)$ (Vergleich zweier Zahlen) erkennen, ob sein lokal zwischengespeicherter Horizont-Subgraph noch gültig ist, und nur im Falle einer Änderung eine vollständige Neuübertragung von G_s^h anfordern (If-None-Match-Mechanismus gemäß [2]). Dies überträgt das in Abschnitt 3 (Unterabschnitt „Zustandslosigkeit und SC-REST“) beschriebene Prinzip der ETag-basierten Versionsprüfung von einzelnen Ressourcenzuständen auf den *gesamten, im Horizont enthaltenen Navigationsgraphen* – ein Client kann damit nicht nur erkennen, ob sich *seine* Ressource geändert hat, sondern auch, ob sich die für ihn relevanten *Spielregeln* (δ eingeschränkt auf G_s^h) geändert haben.

7.8 HST als Architekturstil: Generalisierung von REST

Fielding definiert Architekturstile in [1] durch sukzessives Hinzufügen von Beschränkungen (*constraints*) zu einem „Nullstil“ ohne jede Beschränkung; REST entsteht durch Hinzufügen von Client-Server, Zustandslosigkeit, Cache, Uniform Interface, Layered System und (optional) Code-on-Demand. Die in Abschnitt 7.5 bewiesene lokale Vollständigkeit motiviert eine *zusätzliche* Beschränkung, mit der sich HST formal als echte Verfeinerung von REST einordnen lässt.

Definition 17 (HST-Architekturstil). Der *HST-Architekturstil* ist der REST-Architekturstil [1], erweitert um die folgende zusätzliche Beschränkung:

Horizont-Lokalität. Es existiert ein anwendungsweit oder ressourcenspezifisch konfigurierter Horizont $h \geq 1$, sodass jede Repräsentation einer zustandsbehafteten Ressource zusätzlich zu den unmittelbar gültigen Übergängen (Uniform Interface / HATEOAS) den vollständigen h -Schritt-Horizont-Subgraphen G_s^h gemäß Definition 12 enthält, sodass jeder Client für Aufrufsequenzen der Länge $\leq h$ lokale Vollständigkeit im Sinne von Satz 9 besitzt.

Bemerkung 17. Die Einordnung von HST als *Erweiterung*, nicht als Ersetzung, von REST entspricht Fieldings Methodik [1], Architekturstile inkrementell durch Hinzufügen von Beschränkungen zu definieren. Jede HST-konforme Architektur ist insbesondere REST-konform – Satz 8 zeigt dies explizit für die Zustandslosigkeits-Beschränkung, die unter allen übrigen Beschränkungen invariant bleibt. HST ist somit, ebenso wie SC-REST, kein alternativer Stil zu REST, sondern eine *Verfeinerung*: SC-REST verfeinert REST um Sequenzkonformität (Abschnitt 3), und HST verfeinert SC-REST zusätzlich um Horizont-Lokalität. Die Beziehung

$$\text{REST} \supseteq \text{SC-REST} \supseteq \text{HST}$$

(als Mengen von Architekturen, die die jeweiligen Beschränkungen erfüllen) ist gemäß Bemerkung 11 sogar eine Gleichheit zwischen SC-REST und HST für $h = 1$: SC-REST ist also nicht nur *enthalten* in HST, sondern der Grenzfall $h = 1$ *ist* HST.

Tabelle 2: Die REST-Beschränkungen nach [1] und ihre Realisierung in SC-REST ($h = 1$) bzw. HST ($h \geq 1$ allgemein).

REST-Beschränkung	SC-REST ($h = 1$)	HST ($h \geq 1$, allgemein)
Client-Server	unverändert	unverändert
Zustandslosigkeit	$\pi(id) = s$ als Ressourcenzustand (Abschn. 3)	unverändert gültig für beliebiges h (Satz 8)
Cache	Standard-HTTP-Caching pro Ressourcenzustand	zusätzlich signaturbasierte Kohärenz für G_s^h (Abschn. 7.7)
Uniform Interface (HATEOAS)	Links auf $\Sigma(\delta(s, \sigma))$, Horizont 1	vollständiger Horizont-Subgraph G_s^h , Horizont $h \geq 1$ konfigurierbar
Layered System	unverändert	unverändert
Code-on-Demand	nicht betrachtet	nicht betrachtet
(neu) Horizont-Lokalität	trivial erfüllt für Sequenzlänge ≤ 1	erfüllt für Sequenzlänge $\leq h$ (Satz 9)

7.9 Referenzimplementierung: Berechnung des Horizont-Subgraphen

Listing 5 erweitert die Middleware aus Listing 3/4 um eine Breitensuche, die den Horizont-Subgraphen G_s^h gemäß Definition 12 berechnet, und um eine Funktion, die diesen Subgraphen gemäß Definition 13 in die HST-Repräsentation einbettet.

Listing 5: Breitensuchbasierte Berechnung des Horizont-Subgraphen G_s^h und Einbettung in die HST-Repraesentation

```

1 def horizon_subgraph(process: ProcessAutomaton,
2     s: str, h: int):
3     """
4     Berechnet  $G_s^h = (V, E)$  gemäß Definition
5     "h-Schritt-Horizont-Subgraph" per Breitensuche.
6
7     V: Menge erreichbarer Zustände (Knoten)
8     E: Liste von (s1, operation, s2)-Tripeln (Kanten)
9     """
10    V = {s}
11    E = []
12    frontier = {s}
13
14    for _ in range(h):
15        next_frontier = set()
16        for s1 in frontier:
17            for op in process.allowed_operations(s1):
18                s2 = process.delta[(s1, op)]

```



```

19         E.append((s1, op, s2))
20         if s2 not in V:
21             V.add(s2)
22             next_frontier.add(s2)
23         frontier = next_frontier
24         if not frontier:
25             break # Horizont reicht bis F oder Sackgasse
26
27     return V, E
28
29
30 def sst_representation(process: ProcessAutomaton,
31                       instance_id: str, h: int = 2):
32     """
33     Erstellt die HST-Repraesentation R(id, s, h) gemaess
34     Definition "HST-Repraesentation".
35     """
36     s = process.pi[instance_id]
37     V, E = horizon_subgraph(process, s, h)
38
39     return {
40         "orderId": instance_id,
41         "state": s,
42         "_horizon": h,
43         "_graph": {
44             "nodes": sorted(V),
45             "edges": [
46                 {"from": s1, "op": op, "to": s2}
47                 for (s1, op, s2) in E
48             ],
49         },
50         # h=1-Teilmenge entspricht den bisherigen
51         # HATEOAS-Links (Listing 4, Bemerkung "SC-REST
52         # ist Spezialfall h=1 von HST")
53         "_links": hateoas_links(process, s, instance_id),
54     }

```

Bemerkung 18. Die Laufzeit von `horizon_subgraph` ist $O(|V_s^h| \cdot d) = O(d^{h+1})$ (Lemma 2), da jede der h BFS-Schichten höchstens $|V_s^h|$ Knoten mit jeweils höchstens d ausgehenden Operationen betrachtet; bei Hashtabellen-basierter Implementierung von δ (Satz 2) ist jeder Zugriff $\delta[(s_1, \sigma)]$ in erwarteter konstanter Zeit möglich. Für die in Abbildung 6 betrachteten praxisrelevanten Horizonte $h \leq 4$ und $d \leq 3$ ist dieser Aufwand vernachlässigbar gegenüber der ohnehin anfallenden Fachlogik-Ausführung in Definition 14.

8 Zusammenfassung

Diese Arbeit hat mit SC-REST ein formales Pattern für sequenzgebundene REST-Schnittstellen entwickelt (Abschnitte 3–6) und dieses Pattern in Abschnitt 7 zu *HST* (*Horizon State Transfer*) weiterentwickelt: SC-REST übermittelt pro Antwort die im aktuellen Zustand zulässigen Folgeoperationen (1-Schritt- Horizont), während HST den

vollständigen h -Schritt-Horizont-Subgraphen G_s^h überträgt und damit lokale Vollständigkeit für Aufrufsequenzen der Länge $\leq h$ herstellt (Satz 9), ohne die Zustandslosigkeit nach Fielding [1] zu beeinträchtigen (Satz 8). SC-REST ist dabei formal der Spezialfall $h = 1$ von HST (Bemerkung 11); beide Pattern teilen daher dieselbe Anwendungsgrundlage, die im Folgenden zusammengefasst wird, bevor in Abschnitt 8.2 – nun unter besonderer Berücksichtigung von HST – Erweiterungen und Forschungsrichtungen diskutiert werden.

8.1 Anwendungen

Das SC-REST-Pattern (und, mit größerem Horizont, HST) eignet sich insbesondere für:

- E-Commerce-Checkout-Prozesse (Warenkorb, Versand, Zahlung, Bestätigung) wie in Beispiel 1,
- mehrstufige Antrags- und Genehmigungsprozesse in Verwaltung und Finanzwesen (Einreichung, Prüfung, Freigabe, Auszahlung),
- Onboarding-Flows mit zwingender Reihenfolge (Identitätsprüfung vor Vertragsabschluss vor Zahlungseinrichtung),
- zustandsbehaftete Geräteprotokolle, die über REST gekapselt werden (Initialisierung, Kalibrierung, Messung, Abschluss),
- Multi-Step-Formulare und Wizards in Web-Anwendungen, bei denen Schrittreihenfolgen serverseitig erzwungen werden müssen, um inkonsistente Zwischenzustände zu verhindern.

Für HST (Abschnitt 7) treten insbesondere solche Anwendungen hinzu, bei denen *Planungsvorsprung* fachlich wertvoll ist: Mobile Clients mit instabiler Verbindung (großes β in Definition 15, daher größeres h^* nach Satz 10), grafische Prozess-Editoren und Wizard-UIs, die dem Nutzer *vorab* alle erreichbaren Folgeschritte mehrerer Ebenen anzeigen möchten („Fortschrittsanzeige“ als direkte UI-Repräsentation von G_s^h), sowie – gemäß Bemerkung 14 – Sammelzustände mit permutierbaren Teilschritten, für die HST mit $h \geq r$ die in Abschnitt 4.4 entwickelte signaturbasierte Konformitätsprüfung vollständig auf den Client verlagert.

8.2 Ausblick

Das in dieser Arbeit vorgestellte SC-REST-Pattern und seine Verallgemeinerung zu HST (Abschnitt 7) bilden die formale Grundlage für zahlreiche Erweiterungen und Forschungsrichtungen, die im Folgenden ausführlich diskutiert werden. Die ersten zwölf Unterabschnitte betreffen primär SC-REST und seine in Abschnitt 4 bis 6 entwickelten Resultate; die abschließenden Unterabschnitte widmen sich speziell den durch HST eröffneten Fragestellungen.

8.2.1 Formale Spezifikationssprache für Prozessautomaten

Die Definitionen 2–4 legen nahe, δ nicht imperativ (wie in Listing 4) zu kodieren, sondern *deklarativ* in einer maschinenlesbaren Spezifikation. Ein naheliegender Ansatz ist eine Erweiterung von OpenAPI um ein Feld **x-process-state**, das pro Operation den Vorbedingungszustand (**requires-state**) und den Folgezustand (**transitions-to**) angibt.

Aus einer solchen Spezifikation ließen sich automatisch (1) die Hashtabelle δ für Satz 2, (2) Client-SDKs mit typsicheren Zustandsübergängen, (3) interaktive API-Dokumentation mit Zustandsdiagrammen analog zu Abbildung 1, und (4) Testsuiten generieren, die systematisch alle $\delta(s, \sigma)$ sowie alle erwarteten 409-Fälle (alle $(s, \sigma) \notin \text{dom}(\delta)$) abdecken. Eine solche DSL würde die in Bemerkung 8 geforderte Begrenzung $d(s) \leq D_{\max}$ bereits zur Designzeit statisch prüfbar machen.

8.2.2 Verteilte Prozessinstanzen und das Saga-Pattern

Reale Geschäftsprozesse erstrecken sich häufig über mehrere Microservices, von denen jeder nur einen Teil von Σ implementiert. Eine natürliche Erweiterung besteht darin, den globalen Prozessautomaten P als *Choreographie* mehrerer lokaler Automaten $P_1, \dots, P_n$ zu modellieren, deren Zustände S_i über Ereignisse synchronisiert werden – eine Verbindung zum klassischen Saga-Pattern für verteilte Transaktionen [9]. Im Unterschied zu klassischen Sagas, die primär *Kompensationsaktionen* bei Fehlern definieren, würde eine SC-REST-Choreographie zusätzlich *präventiv* verhindern, dass ein Client überhaupt eine Operation auslöst, deren Vorbedingungen in einem anderen Service noch nicht erfüllt sind. Hierzu wäre Satz 1 auf ein verteiltes Setting zu verallgemeinern, in dem $\pi(id)$ nicht von einem einzelnen Server, sondern von einer verteilten, eventuell replizierten Datenstruktur verwaltet wird – mit den entsprechenden Konsistenzfragen (siehe Punkt „Multi-Tenant“ weiter unten).

8.2.3 Nichtdeterministische Automaten und parallele Teilprozesse

Definition 2 fordert, dass δ eine *Funktion* ist (deterministisch). Viele Prozesse enthalten jedoch *parallele* Zweige, die unabhängig voneinander und potenziell gleichzeitig fortschreiten (im Unterschied zu den in Abschnitt 4.4 behandelten *sequentiell, aber in beliebiger Reihenfolge* auszuführenden Teilschritten). Eine natürliche Erweiterung ist der Übergang von Prozessautomaten zu *Petri-Netzen* [13]: Stellen (places) modellieren Teilprozesse, Marken (tokens) den Fortschritt, und Transitionen werden erst „feuerbar“, wenn alle Vorbedingungs-Stellen markiert sind. Die Konformitätsprüfung (Satz 4) würde dann zu einer Erreichbarkeitsprüfung im Markierungsgraphen des Petri-Netzes, deren Komplexität im Allgemeinen deutlich höher ist als $\Theta(k)$ – ein Forschungsfeld, in dem die Übertragung der Subgraph-Algorithmus-Technik aus Satz 5 auf Markierungsvektoren ein vielversprechender Ansatzpunkt für zukünftige Arbeiten ist.

8.2.4 Sicherheit: Geschützte Zustandstoken

In der hier vorgestellten Form wird $\pi(id)$ serverseitig gespeichert. In hochskalierbaren oder Edge-Architekturen kann es vorteilhaft sein, den Zustand s direkt im Client zu halten – kodiert als signiertes JSON Web Token (JWT), das bei jeder Anfrage mitgesendet wird. Dies würde die Statelessness im klassischen Sinn vollständig wiederherstellen (*kein* serverseitiger Zustand pro Instanz), erfordert jedoch eine kryptografische Signatur, die Manipulation des Zustands durch den Client ausschließt (sonst könnte ein Client durch direktes Setzen von $s := s_4$ die Prüfung $\delta(s, \sigma)$ umgehen). Die formale Korrektheitsaussage aus Satz 1 bliebe in diesem Modell gültig, sofern die Signaturprüfung als zusätzliche, der eigentlichen δ -Prüfung vorgeschaltete Operation modelliert wird, deren Fehlschlagen ebenfalls zu einer (jedoch von *sequence-violation* zu unterscheidenden, 401/403-artigen) Fehlerantwort führt.

8.2.5 Adaptive Vorhersage mittels maschinellem Lernen

Das Latenzmodell aus Definition 10 und Satz 6 geht von einem festen, durch $N(s)$ gegebenen Kandidatensatz für das Prefetching aus. Bei hohem Verzweigungsgrad $d(s)$ (Bemerkung 8) ist es jedoch ineffizient, *alle* $|N(s)|$ Kandidaten gleich zu priorisieren. Stattdessen könnte ein Markov-Modell höherer Ordnung – basierend auf der Historie vorheriger Prozessinstanzen – eine Wahrscheinlichkeitsverteilung $P(\sigma \mid s, \text{Kontext})$ über die nächste Operation schätzen und das Prefetching nur für die w wahrscheinlichsten Kandidaten durchführen (*Top- w -Prefetching*). Satz 6 ließe sich dann zu einem *erwarteten* Latenzgewinn $E[\Delta] = \sum_{\sigma \in \Sigma(s)} P(\sigma \mid s) \cdot \min(T_{\text{think}}, T_{\text{prep}}(\delta(s, \sigma)))$ verfeinern, und die Wahl von w würde einen expliziten Kompromiss zwischen erwartetem Latenzgewinn und Ressourcenverbrauch (Bemerkung 8) parametrisieren. Die in Abbildung 4 gezeigte Lernkurve liefert hierfür bereits die empirische Grundlage: Die mit zunehmender Prozessreife sinkende Fehlerrate $p(n)$ entspricht einer zunehmend „spitzeren“ Verteilung $P(\sigma \mid s)$.

8.2.6 Verifikation und Model Checking

Da $P = (S, \Sigma, \delta, s_0, F)$ ein endlicher Automat ist, lassen sich Eigenschaften wie „jeder Zustand $s \in S \setminus F$ besitzt mindestens einen Pfad nach F “ (*Lebendigkeit*, keine „Sackgassen“) oder „ F ist von s_0 aus stets erreichbar“ mit Standardverfahren der Erreichbarkeitsanalyse in $O(|S| + |E|)$ prüfen (Tiefen- bzw. Breitensuche auf G_P , Definition 7). Für die in Abschnitt 4.4 diskutierten Sammelzustände mit ihrer impliziten 2^r -Struktur (Lemma 1) wären jedoch symbolische Verfahren (BDD-basiertes Model Checking, LTL/CTL-Spezifikationen) notwendig, um Eigenschaften wie „jede Reihenfolge der r optionalen Teilschritte führt zu einem konsistenten Folgezustand“ ohne explizite Aufzählung aller 2^r Zwischenzustände zu verifizieren – ein direkter Anknüpfungspunkt für die formale Methode dieser Arbeit.

8.2.7 Integration mit GraphQL, gRPC und AsyncAPI

Das SC-REST-Pattern wurde für REST/HTTP mit JSON-Repräsentationen formuliert. Die zugrundeliegenden Definitionen 2–6 sind jedoch protokollunabhängig: Für GraphQL ließe sich Σ als Menge von Mutationen interpretieren, wobei 409-artige Fehler als typisierte GraphQL-Errors mit einem analogen `sequence-violation`-Feld kodiert würden; für gRPC entsprächen die Operationen RPC-Methoden, und Sequenzverletzungen würden über den `FAILED_PRECONDITION`-Statuscode signalisiert, dessen Metadaten die Menge $\Sigma(s)$ trügen. Für ereignisgetriebene Architekturen (AsyncAPI) wäre Σ die Menge der konsumierten Nachrichtentypen, und eine Sequenzverletzung würde als *Dead-Letter*-Ereignis mit Verweis auf $\Sigma(s)$ veröffentlicht. In allen Fällen blieben Satz 1 (Korrektheit) und Satz 2 (Laufzeit) unverändert gültig, da beide ausschließlich auf der abstrakten Struktur von P , nicht auf HTTP-spezifischen Details, beruhen.

8.2.8 Hierarchische Automaten und Subprozesse

Komplexe Prozesse lassen sich häufig in Subprozesse zerlegen, die selbst wieder Prozessautomaten gemäß Definition 2 sind. Ein Zustand $s \in S$ des übergeordneten Automaten könnte dann „intern“ einen vollständigen Subprozessautomaten $P_s = (S_s, \Sigma_s, \delta_s, s_0^{(s)}, F_s)$ repräsentieren, wobei der Übergang aus s im übergeordneten Automaten erst erfolgt, wenn P_s einen Zustand in F_s erreicht hat. Der Prozessgraph G_P (Definition 7) würde dadurch zu einem *hierarchischen Graphen*, in dem jeder „Superknoten“ selbst einen Graphen G_{P_s}

enthält. Die Konformitätsprüfung einer Aufrufsequenz würde rekursiv: Auf jeder Hierarchieebene wird Satz 4 angewendet, und für Sammelzustände mit permutierbaren Teilschritten (Abschnitt 4.4) auf jeder Ebene unabhängig Satz 5. Dies eröffnet die Möglichkeit, den Subgraph Algorithmus aus [12] *rekursiv* auf verschachtelte Subgraphen anzuwenden – ein direkter struktureller Bezug zur in [12] diskutierten AST-Analyse, bei der Teilbäume (Subgraphen) ebenfalls rekursiv verglichen werden.

8.2.9 Standardisierung: Vorschlag für eine OpenAPI-Erweiterung „x-sc-rest“

Die in dieser Arbeit entwickelten Definitionen könnten Grundlage eines formalen Standardisierungsvorschlags sein, etwa als OpenAPI-Erweiterungsfeld **x-sc-rest** oder als eigenständiges Internet-Draft analog zu RFC 7807 [3]. Ein solcher Standard müsste mindestens spezifizieren: (1) das Format zur Deklaration von $S, \Sigma, \delta, s_0, F$ (vgl. „Formale Spezifikationssprache“ oben), (2) den Problem-Detail-Typ **sequence-violation** inklusive des Feldes **allowedOperations**, (3) die HATEOAS-Link-Relationstypen für Folgeoperationen, und (4) optionale Erweiterungsfelder für Sammelzustände (Definition 8), etwa **x-sc-rest-collection** mit den Feldern **operations** (entsprechend Σ_{opt}) und **completion** (entsprechend σ_{done}). Ein solcher Standard würde die in Pautasso et al. [14] diskutierte Debatte zwischen „RESTful“ und „Big Web Services“ (WS-BPEL) um eine dritte Position ergänzen: prozessbewusste REST-APIs, die die Vorteile beider Welten – die Einfachheit von REST und die Prozessgarantien von WS-BPEL-Choreographien – formal vereinen.

8.2.10 Experimentelle Validierung in Produktionssystemen

Die in Abschnitt 5 gezeigten Diagramme basieren auf simulierten Daten. Eine naheliegende Folgearbeit ist die Instrumentierung einer SC-REST-Referenzimplementierung (Abschnitt 6) in einem produktiven Microservice-Umfeld [7], um (1) die in Satz 6 postulierte Latenzreduktion unter realer Lastverteilung von T_{think} und T_{prep} zu vermessen, (2) die tatsächliche Verteilung der Verzweigungsgrade $d(s)$ in realen Prozessmodellen zu erheben (als Eingabe für Bemerkung 8), und (3) die in Lemma 1 postulierte Zustandsraum-Explosion für reale Sammelzustände (typische Werte für r) zu quantifizieren. Eine offene Benchmark-Suite, die verschiedene Prozessautomaten P unterschiedlicher Größe $|S|$, $|\Sigma|$ und Verzweigungsgrade enthält, würde zudem den systematischen Vergleich verschiedener Implementierungsstrategien für δ (Hashtabelle, Entscheidungsbaum, kompilierter endlicher Automat) ermöglichen.

8.2.11 Energieeffizienz durch reduzierte Fehlversuche

Jede 409-Antwort gemäß Satz 1(c) repräsentiert eine „vergebliche“ Anfrage: Netzwerkübertragung, TLS-Handshake (falls keine Connection wiederverwendet wird), Serverseitige Verarbeitung der Anfrage und der Fehlerantwort – alles ohne fachlichen Nutzen. Die in Abbildung 4 gezeigte sinkende Fehlerrate $p(n)$ impliziert daher nicht nur einen Latenz-, sondern auch einen Energieeffizienzgewinn: Pro vermiedener 409-Antwort wird die gesamte Verarbeitungskette einmal weniger durchlaufen. Eine quantitative Abschätzung dieses Effekts – etwa in Joule pro vermiedener Anfrage, aggregiert über Rechenzentren mit hohem API-Durchsatz – wäre ein Beitrag zur Diskussion um „Green Software Engineering“ und ergänzt die in Bemerkung 8 diskutierte Kostenseite des Prefetchings: Während Prefetching gezielt *zusätzliche* Berechnungen vorzieht, um Latenz zu sparen, spart die Sequenzkontrolle

selbst Berechnungen *vollständig* ein, indem sie von vornherein unmögliche Anfragen früh und billig (Satz 2, $O(1)$) abweist.

8.2.12 Multi-Tenant-Architekturen und verteilte Zustandsspeicher

Definition 5 setzt voraus, dass $\pi : \text{ID} \rightarrow S$ konsistent gelesen und geschrieben werden kann. In horizontal skalierten Systemen wird π typischerweise in einem verteilten Schlüssel-Wert-Speicher (z. B. Redis-Cluster) gehalten. Damit stellt sich die Frage, welche Konsistenzgarantien für π erforderlich sind, damit Satz 1 weiterhin gilt: Wird $\pi(id)$ unter Eventual Consistency repliziert, könnten zwei nahezu gleichzeitige Anfragen für dieselbe Prozessinstanz id auf unterschiedlichen Replikaten denselben Ausgangszustand s sehen und beide einen (jeweils für sich konformen) Übergang $\delta(s, \sigma)$ und $\delta(s, \sigma')$ ausführen – ein klassisches *Lost-Update*-Problem, das die Eindeutigkeit des Laufs aus Definition 3 verletzt. Eine Erweiterung dieser Arbeit müsste daher entweder (a) starke Konsistenz für $\pi(id)$ fordern (z. B. mittels eines verteilten Locks oder einer Single-Writer-Partitionierung nach id , was angesichts der CAP-Theorem-Implikationen Verfügbarkeitseinbußen bei Partitionierung bedeutet), oder (b) Satz 1 zu einer *optimistischen* Variante verfeinern, bei der konkurrierende Übergänge mittels eines Versionszählers (analog zu ETag/ If-Match) erkannt und einer der beiden Übergänge nachträglich mit 409 Conflict abgewiesen wird – eine Idempotenz- und Konfliktbehandlungsstrategie, die eng mit der in Abschnitt 4.6 diskutierten Replay-Sicherheit (Satz 7) zusammenhängt und in [10] für verteilte Transaktionen allgemein diskutiert wird.

8.2.13 Adaptiver Horizont: Lernen von h^* pro Client und Prozesstyp

Satz 10 liefert h^* als Funktion der Parameter α, β, d, p – in der Praxis sind insbesondere β (Round-Trip-Latenz) und p (Fortsetzungswahrscheinlichkeit) jedoch nicht global konstant, sondern hängen vom *Client* (mobile Verbindung vs. Rechenzentrums-zu-Rechenzentrums-Aufruf, also unterschiedliches β) und vom *Prozesstyp* bzw. sogar vom *individuellen Nutzer* (manche Nutzer brechen Bestellvorgänge häufig nach dem ersten Schritt ab, andere durchlaufen sie zuverlässig bis zum Ende, also unterschiedliches p) ab. Dies setzt die in Abschnitt 8.2 (Unterabschnitt „Adaptive Vorhersage mittels maschinellem Lernen“) für T_{think} und T_{prep} skizzierte Idee fort: Ein lernendes System könnte $\hat{\beta}$ aus den RTT-Werten der TCP-Verbindung des jeweiligen Clients und $\hat{p}$ aus der historischen Sequenzlängenverteilung des jeweiligen Nutzers (oder, bei neuen Nutzern, aus einem Prozesstyp-Prior) schätzen und daraus gemäß der geschlossenen Form in Satz 10(c) einen *personalisierten* Horizont $\hat{h}^*(\text{client}, \text{user})$ berechnen, der pro Anfrage als ?horizon=-Parameter (Definition 14) gesetzt wird. Im Gegensatz zu den meisten ML-Einsatzfeldern in zustandsbehafteten APIs ist diese Anwendung *folgenlos im Fehlerfall*: Eine falsche Schätzung von $\hat{h}^*$ führt nach Satz 8 höchstens zu suboptimalen Kosten $C(\hat{h}^*) > C(h^*)$, niemals zu einer inkorrekten oder inkonsistenten Antwort – das Lernproblem ist also ein reines Optimierungs-, kein Korrektheitsproblem, was den Einsatz von ML hier mit deutlich geringerem Risiko behaftet als etwa bei der in [13]-artigen Systemen üblichen Zustandsvorhersage.

8.2.14 Informationsfreigabe und Zugriffskontrolle im Horizont-Subgraphen

Definition 13 setzt implizit voraus, dass *der gesamte* Horizont-Subgraph G_s^h für den anfragenden Client offenlegbar ist. In Prozessen mit rollenbasierter Autorisierung – etwa Genehmigungs-Workflows, in denen ein Sachbearbeiter nur die für seine Rolle bestimmten

Übergänge sehen darf, während Eskalationspfade (z. B. `POST /reject-and-escalate`) nur für Vorgesetzte sichtbar sein sollen – würde eine ungefilterte Übertragung von G_s^h dem Client *Wissen über die Struktur des Prozessautomaten* offenlegen, das über seine eigenen Handlungsmöglichkeiten hinausgeht: Allein die *Existenz* eines Eskalationszustands im Horizont könnte ein Informationsleck darstellen (etwa wenn die bloße Existenz eines „Sonderkonditionen“-Zustands für reguläre Kunden vertraulich sein soll). Eine konsequente Erweiterung von Definition 14 wäre daher eine *rollenfilterte* Variante

$$G_s^h \upharpoonright_\rho := (V_s^h \cap \text{sichtbar}(\rho), E_s^h \cap (\text{sichtbar}(\rho) \times \Sigma \times \text{sichtbar}(\rho))),$$

wobei ρ die Rolle des Clients und $\text{sichtbar}(\rho) \subseteq S$ die für ρ sichtbaren Zustände bezeichnet. Eine formale Folgearbeit müsste klären, ob $G_s^h \upharpoonright_\rho$ weiterhin die in Satz 9 gezeigte lokale Vollständigkeit für die dem Client tatsächlich erlaubten Operationen besitzt – vermutlich ja, sofern $\text{sichtbar}(\rho)$ unter δ für die Operationen aus $\Sigma_\rho \subseteq \Sigma$ abgeschlossen ist – und wie sich dies auf die in Definition 16 definierte Signatur auswirkt (vermutlich müsste $\text{Sig}(G_s^h \upharpoonright_\rho)$ rollenabhängig berechnet werden, was die Cache-Kohärenz aus Abschnitt 7.7 um eine zusätzliche Dimension – die Rolle – erweitert).

8.2.15 Komposition zu einem globalen Anwendungsgraphen

Reale Anwendungen bestehen aus mehreren Prozessautomaten $P_1, \dots, P_n$ – etwa einem Bestellprozess (Beispiel 1), einem Retourenprozess und einem Reklamationsprozess, die über gemeinsame Ressourcen (dieselbe Bestellung) verknüpft sind. Eine natürliche Erweiterung von Definition 12 wäre ein *globaler Anwendungsgraph* $G_{\text{App}} := \bigsqcup_{i=1}^n P_i \cup E_{\text{cross}}$, wobei E_{cross} *Querverweis-Kanten* zwischen Zuständen verschiedener Prozessautomaten enthält (z. B. vom Endzustand $s_5 \in F_{\text{Bestellung}}$ zum Startzustand $s'_0 \in S_{\text{Retoure}}$, gemäß der Operation `POST /orders/{id}/returns`). Der Horizont-Subgraph G_s^h eines Zustands s würde dann gegebenenfalls *Zustände anderer Prozessautomaten* als Randknoten enthalten, sobald h groß genug ist, um E_{cross} -Kanten zu erreichen. Dies würde HATEOAS von der Navigation *innerhalb* eines Prozesses zur Navigation *zwischen* Prozessen verallgemeinern – formal müsste hierzu Lemma 2 auf G_{App} mit einem *kompositionalen* Verzweigungsgrad $d_{\text{App}} = \max(d_1, \dots, d_n, |E_{\text{cross}}|)$ verallgemeinert werden, und Satz 10 müsste klären, ob ein einheitlicher Horizont h^* über Prozessgrenzen hinweg sinnvoll ist oder ob E_{cross} -Kanten mit einem eigenen, typischerweise größeren Kostenfaktor $\alpha_{\text{cross}} > \alpha$ (Cross-Service-Aufruf statt lokaler Tabellenzugriff) zu gewichten sind.

8.2.16 Graphkompression für große Horizonte mittels Signaturen

Für $h \geq r$ (Bemerkung 14) enthält $G_{s_c}^h$ für einen Sammelzustand s_c den vollständigen 2^r -Knoten-Entscheidungsbaum aller Teilmengen $T \subseteq \{1, \dots, r\}$ – nach Lemma 1 potenziell sehr groß. Viele dieser Teilbäume sind jedoch *isomorph*: Der von einem Zwischenzustand s_T aus erreichbare Teilgraph hängt (bei symmetrischen optionalen Teilschritten) nur von $|\{1, \dots, r\} \setminus T|$, der Anzahl noch ausstehender Teilschritte, ab, nicht von T selbst. Eine signaturbasierte Kompression könnte daher isomorphe Teilbäume durch *Verweise* auf eine einzige kanonische Repräsentation ersetzen, identifiziert über die Signatur $\text{Sig}(\cdot)$ aus Definition 16: Statt $\binom{r}{k}$ strukturell identischer Teilbäume für $|T| = k$ würde nur *ein* Teilbaum übertragen und $\binom{r}{k}$ Referenzen darauf. Dies entspricht dem klassischen Prinzip der *strukturellen Teilung* (*structure sharing*) in funktionalen Datenstrukturen, übertragen auf die HST-Repräsentation; eine formale Analyse müsste zeigen, dass diese Kompression die in Bemerkung 13 festgestellte exponentielle Übertragungsgröße $|V_s^h| = O(d^h)$ auf eine

polynomielle Größe der komprimierten Repräsentation reduziert – ein direktes Analogon zur in Bemerkung 7 gezeigten Raumersparnis von $O(2^r)$ auf $O(r)$ für die serverseitige Kodierung, nun angewandt auf die clientseitig übertragene HST-Repräsentation.

8.2.17 Beziehung zu GraphQL: clientseitig verhandelter Horizont

Der in Abschnitt 8.2 (Unterabschnitt „Integration mit GraphQL, gRPC und AsyncAPI“) angesprochene Vergleich mit GraphQL gewinnt durch HST eine neue Dimension: GraphQL erlaubt dem Client, die *Tiefe* verschachtelter Relationen einer Anfrage selbst zu spezifizieren (etwa „lade einen Autor, dessen Bücher, und für jedes Buch dessen Rezensionen“ – eine Tiefe von 2). Der Horizont h in Definition 13 ist strukturell analog: Ein HST-Server könnte h nicht als statische Konfiguration, sondern als *vom Client pro Anfrage spezifizierten Parameter* behandeln (wie bereits in Satz 8 unter Punkt (3) als Option erwähnt), wodurch der Client den in Satz 10 berechneten Tradeoff selbst – abhängig von seiner *eigenen* Einschätzung von β (seiner Verbindungsqualität) und p (seinem geplanten Workflow) – vornehmen könnte, statt sich auf eine serverseitige Schätzung von $\hat{\beta}, \hat{p}$ zu verlassen. Eine GraphQL-artige Anfrage könnte dann etwa lauten: `GET /orders/{id}?horizon=3&fields=state,total`, wobei `horizon=3` den gewünschten h -Wert und `fields` die gewünschten Felder von $\rho(id, s)$ spezifiziert – eine Verschmelzung der „deklarativen Datenanfrage“ von GraphQL mit der „deklarativen Navigationsanfrage“ von HST, die formal als Erweiterung von Definition 13 um einen client-spezifisierten Parameter h (statt eines fest konfigurierten) zu fassen wäre.

8.2.18 Empirische Validierung des Kostenmodells (α, β, p)

Der in Abschnitt 8.2 (Unterabschnitt „Experimentelle Validierung in Produktionssystemen“) für SC-REST skizzierte empirische Validierungsbedarf gilt für HST in verstärktem Maße, da Satz 10 – anders als die asymptotischen Aussagen zu SC-REST – von *konkreten Werten* der Parameter α, β, p, d abhängt. Eine produktive Instrumentierung müsste (1) α aus der tatsächlich gemessenen Serialisierungsgröße von Knoten/Kanten kalibrieren (die in Abbildung 6 angenommenen 250 Byte sind eine grobe Schätzung; reale JSON- oder HAL-Repräsentationen könnten durch Kompression wie gzip oder durch das in Abschnitt 8.2 (Graphkompression) skizzierte *structure sharing* deutlich darunter liegen), (2) β aus der tatsächlichen Verteilung der Round-Trip-Zeiten verschiedener Clientklassen (mobil, Desktop, Service-zu-Service) erheben, und (3) p aus den *tatsächlichen* Sequenzlängenverteilungen unterschiedlicher Prozesstypen schätzen – letzteres ist methodisch eng mit der für Abbildung 4 verwendeten empirischen Fehlerratenkurve $p(n)$ verwandt, jedoch mit umgekehrter Fragestellung: Während $p(n)$ die Wahrscheinlichkeit *eines Sequenzfehlers nach n Trainingsanfragen* misst, beschreibt p in Definition 15 die Wahrscheinlichkeit *einer Fortsetzung* der Sequenz unabhängig von deren Korrektheit. Mit kalibrierten $(\hat{\alpha}, \hat{\beta}, \hat{p})$ ließe sich die in Abbildung 7 gezeigte theoretische Stufenfunktion h^* gegen den empirisch tatsächlich kostenoptimalen Horizont validieren.

8.2.19 Versionsmigration laufender Prozessinstanzen

Satz 8 setzt voraus, dass P während der Bearbeitung einer Anfrage statisch ist; Abschnitt 7.7 behandelt *Erkennung* von Änderungen $P \rightarrow P'$ via Signaturvergleich, jedoch nicht deren *Behandlung* für *bereits laufende* Prozessinstanzen (id, s) mit $s \in S$. Wird P zu P' aktualisiert (etwa weil ein neuer optionaler Teilschritt σ_{r+1} eingeführt oder ein Zustand $s \in S$ aus S' entfernt wird), stellt sich die Frage, wie eine Instanz mit $\pi(id) = s \notin S'$ zu

behandeln ist. Eine formale Behandlung dieses Problems könnte eine *Migrationsabbildung* $\mu : S \rightarrow S'$ definieren, mit den Anforderungen: (a) $\mu(s) = s$ für $s \in S \cap S'$ (unveränderte Zustände bleiben erhalten), (b) für $s \in S \setminus S'$ muss $\mu(s) \in S'$ einen „äquivalenten“ Zustand in P' bezeichnen, wobei Äquivalenz bedeutet, dass die für s in P konformen Restsequenzen (gemäß Definition 4) unter μ auf in P' ab $\mu(s)$ konforme Restsequenzen abgebildet werden – eine Bedingung, die strukturell an die in Satz 9 verwendete Charakterisierung über δ^* erinnert, jedoch *zwischen zwei verschiedenen Automaten* P und P' statt innerhalb eines einzelnen G_s^h . Existiert kein solches $\mu(s)$ (etwa weil ein ganzer Prozesszweig in P' entfernt wurde), müsste die Instanz in einen speziellen Migrationszustand überführt und gegebenenfalls manuell oder durch eine Kompensationslogik (vgl. das in Abschnitt 8.2 erwähnte Saga-Pattern und [9]) abgeschlossen werden. Die in Abschnitt 7.7 beschriebene signaturbasierte Erkennung von $P \rightarrow P'$ wäre damit der *Trigger*, die hier skizzierte Migrationsabbildung μ die *Behandlung* – zusammen ergäben sie ein vollständiges Lebenszyklusmodell für Prozessautomaten, die sich über die Lebensdauer einzelner Instanzen hinweg ändern können.

8.3 Implementierungshinweis

Der vollständige Code der Referenzimplementierung sowie ergänzende Testfälle sind verfügbar unter: <https://gitlab.com/epp-group/screst>.

Literatur

- [1] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Dissertation, University of California, Irvine.
- [2] Fielding, R., Reschke, J. (Hrsg.) (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. RFC 7231, IETF.
- [3] Nottingham, M., Wilde, E. (2016). *Problem Details for HTTP APIs*. RFC 7807, IETF.
- [4] Richardson, L., Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
- [5] Fowler, M. (2010). *Richardson Maturity Model – steps toward the glory of REST*. martinowler.com.
- [6] Webber, J., Parastatidis, S., Robinson, I. (2010). *REST in Practice: Hypermedia and Systems Architecture*. O'Reilly Media.
- [7] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*, 2. Auflage. O'Reilly Media.
- [8] Hohpe, G., Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- [9] Garcia-Molina, H., Salem, K. (1987). *Sagas*. ACM SIGMOD Record, 16(3), S. 249–259.
- [10] Helland, P. (2007). *Life Beyond Distributed Transactions: An Apostate's Opinion*. 3rd Biennial Conference on Innovative Data Systems Research (CIDR).

- [11] Abboud, A., Backurs, A., Vassilevska Williams, V. (2015). *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. IEEE 56th Annual Symposium on Foundations of Computer Science (FOCS).
- [12] Epp, S. (2026). *Der Subgraph Algorithmus*. Eigenverlag / GitLab epp-group.
- [13] Murata, T. (1989). *Petri Nets: Properties, Analysis and Applications*. Proceedings of the IEEE, 77(4), S. 541–580.
- [14] Pautasso, C., Zimmermann, O., Leymann, F. (2008). *RESTful Web Services vs. “Big” Web Services: Making the Right Architectural Decision*. Proceedings of the 17th International Conference on World Wide Web (WWW).